

23CSE203

DATA STRUCTURES AND ALGORITHM

Submitted by	
Name	Venkateswari. G
Roll.no	AV.SC.U4CSE25029
Section	CSE-A

SUBMITTED TO	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR (Sl. Grade)
DEPARTMENT	SCHOOL OF COMPUTING

Credit	5

Index

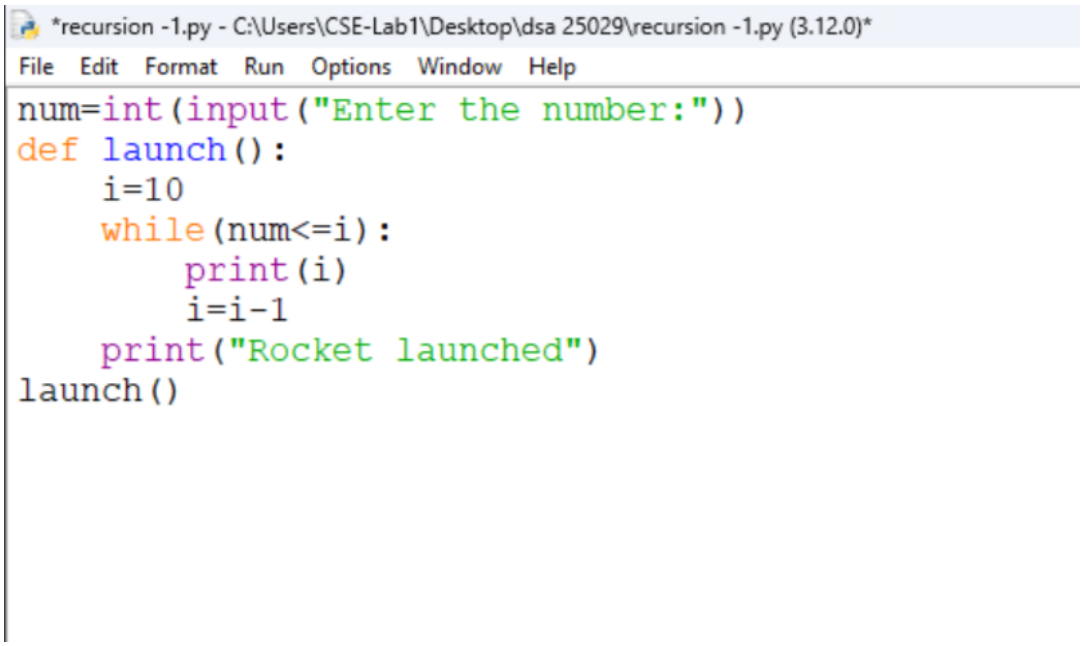
S.no	Title	Page.no	Date
1.	**A)** A rocket launch system displays a countdown before launch. Instead of using loop, implement the countdown using recursion. Write a recursive program to display a countdown from n to 1 followed by launch. **B)** Bank calculates the growth of an investment using compound intrest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n(power), where P=principal growth factor, n=no.of years. **C)** Write a recursive progrsm to print the first n terms of fibanocci series **D)** An hr department stores employee ids in a list develop a recursive program to search for a given employee id. Write a recursive function to search for an employee id in the list. **E)** In a logistics the no.of possible ways to arrange n different parcels calculate using the factorial (n!). develop a recursive program to calculate the factorial of given number.	4-13	23-07-26
2.	**A)** write a python program to perform linear search on the given input **B)** Write a python program to perform binary search on the given input. Verify by taking sorted array and unsorted array as a list.	14-18	30-07-26

3.	**A)** write a python program to sort the elements using a bubble sort technique. Take an input elements from the user. **B)** Insertion sort **C)**selection sort **D)**Merge sort **E)** Quick sort	19-28	06-08-26
4.	**A)**Write a python program to implement a single linked list which would perform the below operations, by taking an input from the user. a. create a linked list b. Insert at beginning. c. Insert at end. d. Insert at a specific index. e. Delete by value. f. Delete 1st node. g. Delete last node. h. Count no. of nodes i. Display. j. Exit. B)Doubly linked list C) circular linked list	29-53	13-08-26
5.	A) stack using array B) stack using linked list	54-59	27-08-26
6.	A) write a python programme to implement queue and its operations such as inqueue,dequeue,peek,display with implement using array. B) implement using linked list. C) implement using circular queue.	60-71	03-09-26
7.	A) Binary tree using list. B) Binary tree using array	72-77	24-09-26
8.	Write a python programme to implement binary search tree with its operation.	78-81	08-10-26
9.	Write a python program to implement avl trees with its operation.	82-86	22-10-26
10.	Python programs for hashing using separate chaining, linear probing, and quadratic probing.	87-92	29-10-26

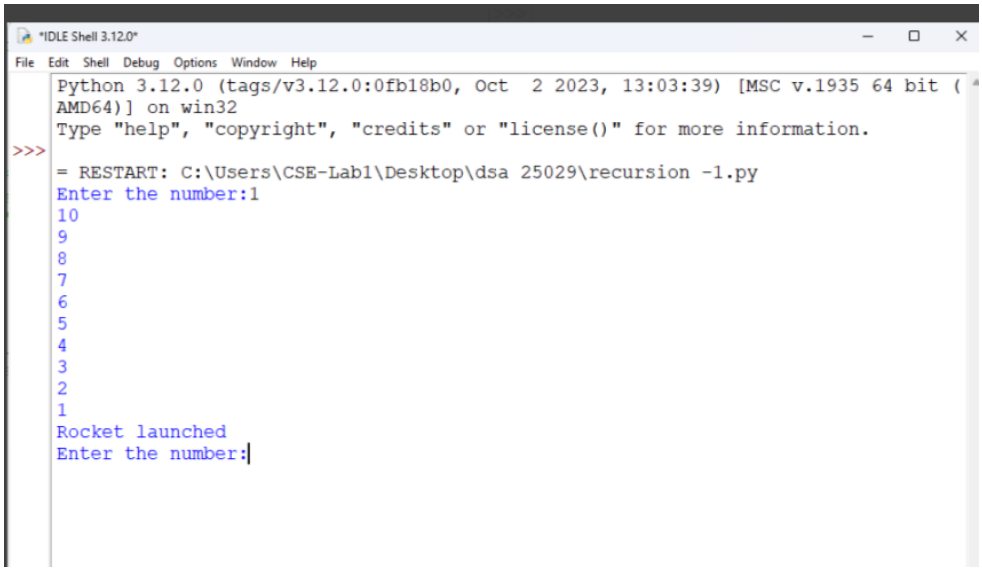
Week-1

A) Aim: A rocket launch system displays a countdown before launch. Instead of using loop, implement the countdown using recursion. Write a recursive program to display a countdown from n to 1 followed by launch.

Program:



Output:



Explanation:

- It takes a number as input from the user and stores it in num.
- It creates a function named launch().
- It starts the countdown from 10 by assigning i = 10.
- It prints the value of i as long as num <= i.
- It decreases the value of i by 1 after each iteration.
- When the loop ends, it prints "Rocket launched" and executes the function using launch().

Error table:

s.no	Error obtained	Rectification
1.	Infinite loop	Iteration at correct place
2.	Calling function at wrong place	Placed with correct syntax

B) Aim: Bank calculates the growth of an investment using compound intrest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n(power), where P=principal growth factor, n=no.of years.

Program:

dsa25029.py - C:/Users/G.venkteswari/AppData/Local/Programs/Python/Python313/dsa250...

File Edit Format Run Options Window Help

```
p = float(input("Enter principal growth factor: "))
n = int(input("Enter number of years: "))

def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)

result = power(p, n)
print("Power =", result)
```

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/G.venkteswari/AppData/Local/Programs/Python/Python313/dsa250
29.py
Enter principal growth factor: 56
Enter number of years: 7
Power = 1727094849536.0
>>>
```

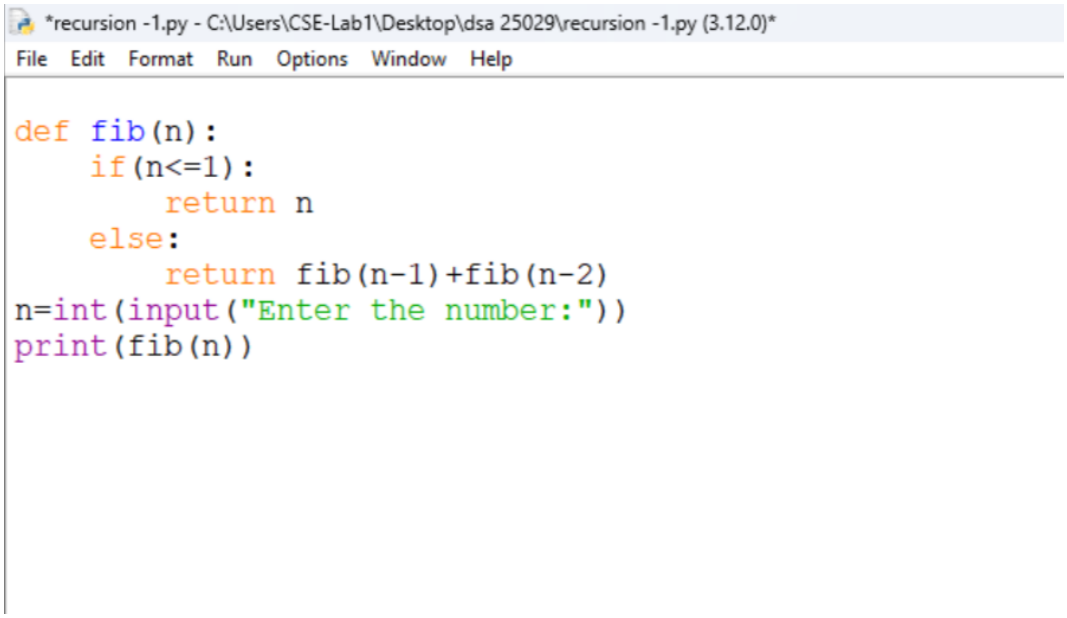
Explanation:

- The program takes the principal growth factor p and number of years n as input.
- It defines a recursive function power(p, n).
- If n is 0, the function returns 1 (base case).
- Otherwise, it returns p * power(p, n-1).
- The recursive calls continue until n becomes 0.
- The final value of p^n is displayed as the output.
- Error table:

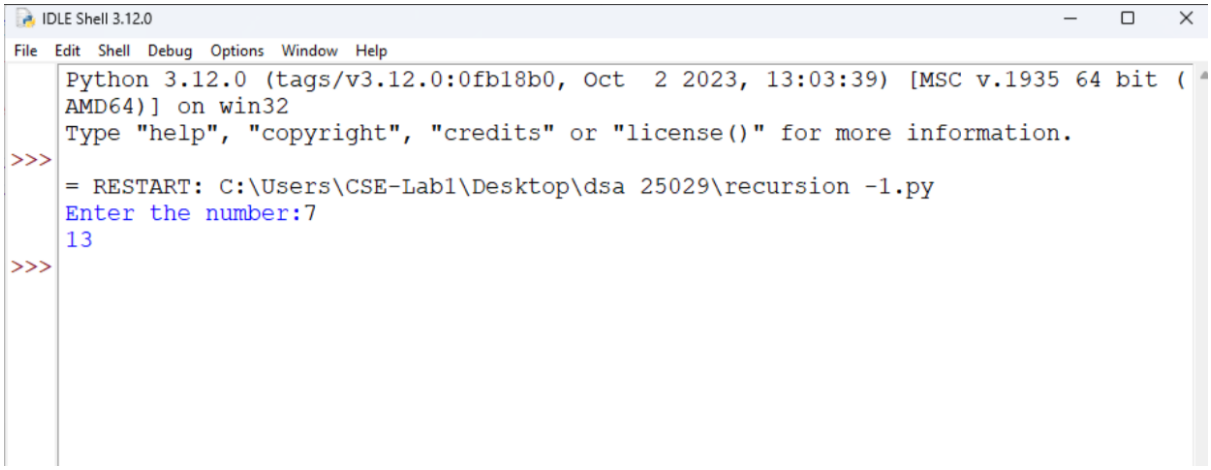
s.no	Error obtained	Rectification
1.	Return 0	Return 1
2.	print	return

C) Aim: Write a recursive progrsm to print the first n terms of fibanocci series

Program:



Output:



Explanation:

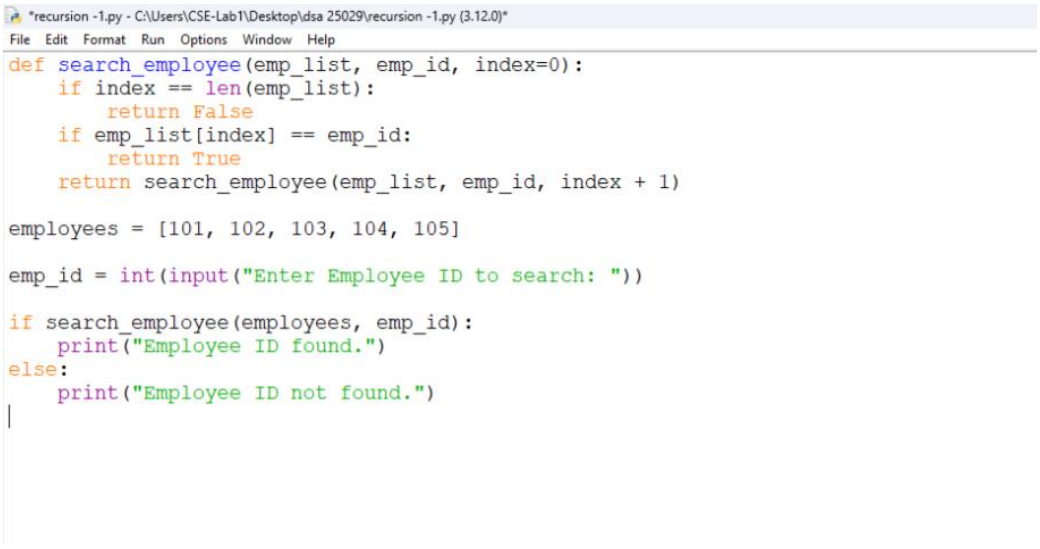
- For entering the number as input from user we defined as n
- We declared variable for formula.
- If input number is less than one then it return “1”
- Or it will goes under fibanocci iteration.

Error:

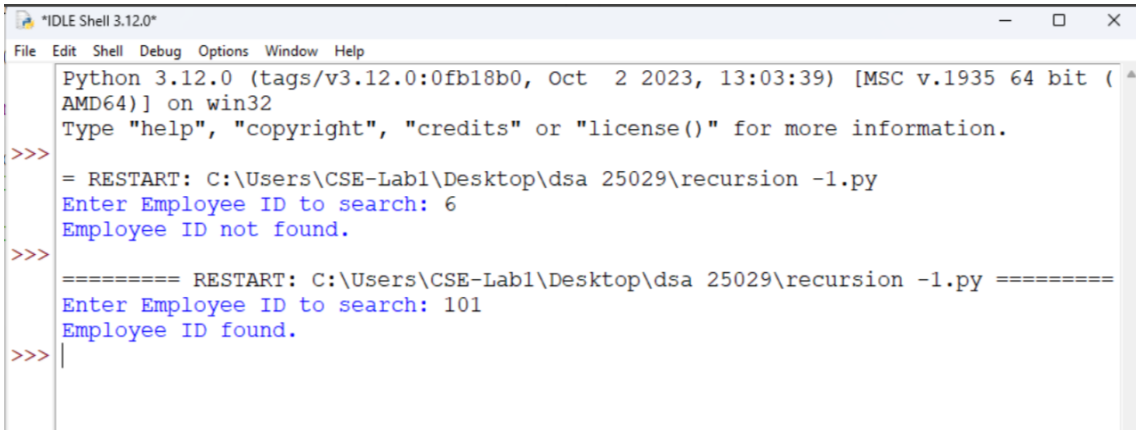
s.no	Error obtained	Rectification
1.	Forgot to declare n<=1	rectified
2.	Formula mistake	rectified

D) Aim: An hr department stores employee ids in a list develop a recursive program to search for a given employee id. Write a recursive function to search for an employee id in the list.

Program:



Output:



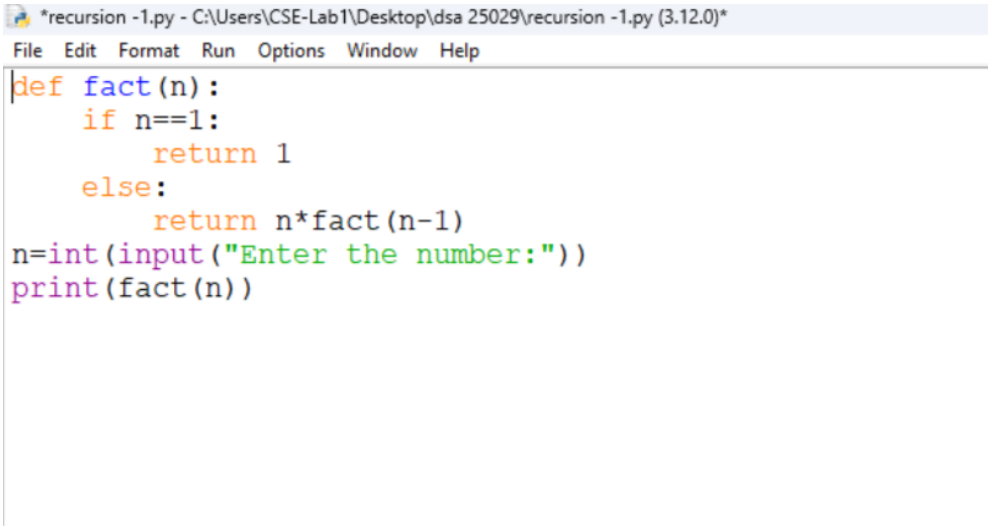
Explanation:

- The program creates a function to search for an employee ID in a list.
- It checks one employee ID at a time, starting from the first ID.
- If the entered employee ID matches the current ID, it returns True.
- If it does not match, the function moves to the next ID using recursion.
- If all employee IDs are checked and no match is found, it returns False.
- Finally, the program displays "Employee ID found." if the ID exists; otherwise, it displays "Employee ID not found."
- Error:

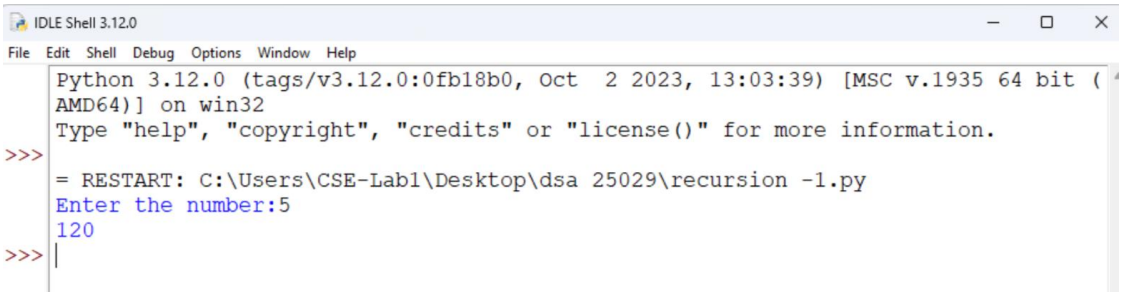
s.no	Error obtained	Rectification
1.	Emp lst	Emp_list
2.	print	Return

E) Aim: In a logistics the no.of possible ways to arrange n different parcels calculate using the factorial (n!). develop a recursive program to calculate the factorial of given number.

Program:



Output:



Explanation:

- The program defines a recursive function fact() to calculate the factorial of a number.
- It takes a number as input from the user.
- If the number is 1, the function returns 1 (base case).
- Otherwise, it multiplies the number by the factorial of the previous number (n-1).
- The recursive calls continue until the number becomes 1.

- Finally, the program prints the factorial of the given number.

Error table:

s.no	Error obtained	Rectification
1.	n+1	n-1
2.	Syntax error	Rectified

WEEK-2

A) AIM: Write a python programm to perfor linear search on the given input.

Program:

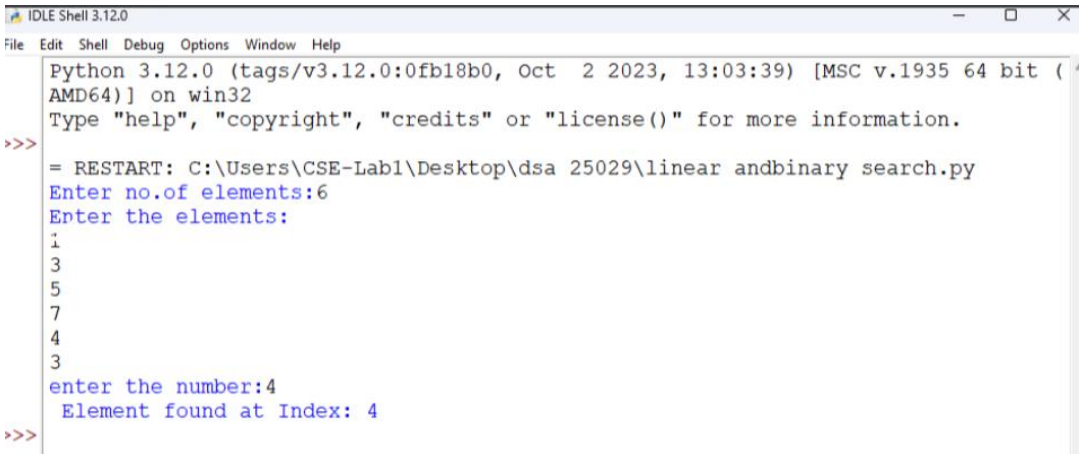
linear andbinary search.py - C:\Users\CSE-Lab1\Desktop\dsa 25029\linear andbinary search.py (3.12.0)

File Edit Format Run Options Window Help

```
#Linear search

n=int(input("Enter no.of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
key=int(input("enter the number:"))
def linear_search(arr,key):
    for i in range(0,len(arr)):
        if arr[i]==key:
            print(" Element found at Index:",i)
            return
    print("Not found")
linear_search(arr,key)
```

Output:



Explanation:

- In linear search we can append elements in sorted or unsorted way also.
- After placing the target we need to check the all the elements in a order.
- The time complixity is O(n).
- The target element may be found at beginning or last or at middle.

Error table:

s.no	Error obtained	Rectification
1.	Return placed in wrong place	Corrected later
2.	Calling the function	Rectified

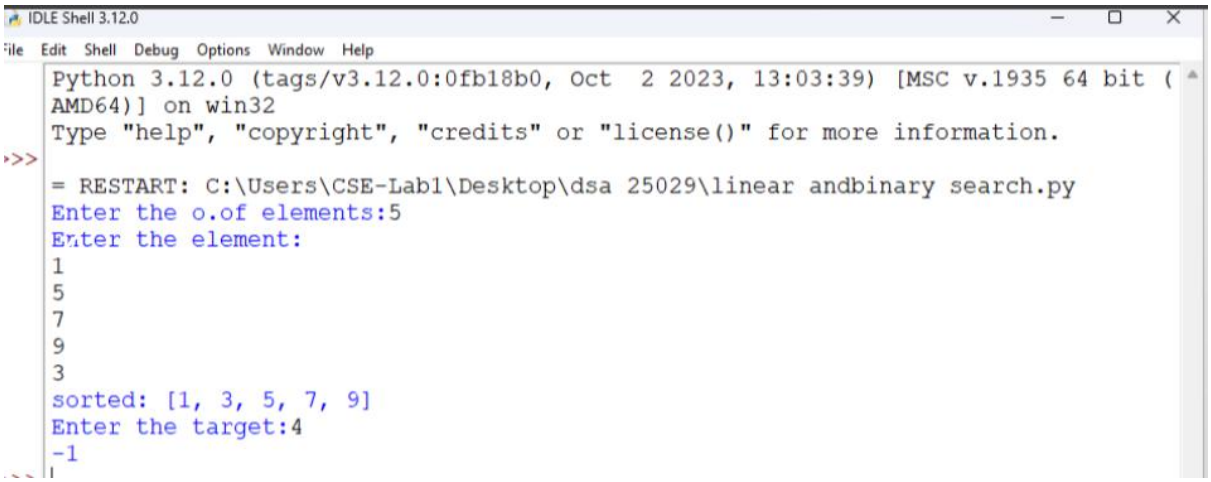
B) Aim: Write a python program to perform binary search on the list.

Program: Unsorted

```
#binary search unsorted

n=int(input("Enter the o.of elements:"))
arr=[]
print("Enter the element:")
for i in range(n):
    arr.append(int(input()))
arr.sort()
print("sorted:",arr)
key=int(input("Enter the target:"))
def binary_search(arr,key):
    low=0
    high=len(arr)-1
    while low<=high:
        mid=(low+high)//2
        if arr[mid]==key:
            return mid
        elif arr[mid]<key:
            low=mid+1
        else:
            high=mid=1
    return -1
print(binary_search(arr,key))
```

Output:



Program: Sorted

linear andbinary search.py - C:\Users\CSE-Lab1\Desktop\dsa 25029\linear andbinary search.py (3.12.0)

File Edit Format Run Options Window Help

```
#binary search sorted
n=int(input("enter the no.of elements:"))
arr=[]
print("Enter the element:")
for i in range(n):
    arr.append(int(input()))
key=int(input("Enter the target:"))
if arr==sorted(arr):
    print("It is sorted")
else:
    print("array is not sorted")
    arr.sort()
    print("sorted array:",arr)
def binary_search(arr,key):
    low=0
    high=len(arr)-1
    while low<=high:
        mid=(low+high)//2
        if arr[mid]==key:
            return 'Element found at index:', mid
        elif arr[mid]<key:
            low=mid+1
        else:
            high=mid-1
    return -1
print(binary_search(arr,key))
```

Output:

IDLE Shell 3.12.0

File Edit Shell Debug Options Window Help

```
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct  2 2023, 13:03:39) [MSC v.1935 64 bit
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\CSE-Lab1\Desktop\dsa 25029\linear andbinary search.py
enter the no.of elements:5
Enter the element:
2
3
5
4
1
Enter the target:5
array is not sorted
sorted array: [1, 2, 3, 4, 5]
('Element found at index:', 4)
```

Explanation:

- The binary search should always executed in sorted order.
- Either it can be ascending or descending order.
- So the program is in unsorted order so we need to do sorting first later the binary search is executed.
- Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

WEEK-3

A)Aim: Write a python programme to sort the elements using a bubble sort technique. Take an input elements from the users.

Programme:

sortings.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/New folder/sortings.py (3.13.7)

File Edit Format Run Options Window Help

```
# Bubble Sort
n = int(input("Enter the number of elements: "))
arr = []
for i in range(n):
    x = int(input(f"Enter element {i + 1}: "))
    arr.append(x)
for i in range(n - 1):
    for j in range(n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
print("Sorted array:", arr)
```

Output:

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/G.venkteswari/OneDrive/Desktop/DSA/New folder/sortings.py =
Enter the number of elements: 5
Enter element 1: 90
Enter element 2: 3
Enter element 3: 45
Enter element 4: 1
Enter element 5: 100
Sorted array: [1, 3, 45, 90, 100]
>>>
```

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\G.venkteswari\OneDrive\Desktop\DSA\sortings\bubble sort.py
Enter the number of elements: 0
Sorted array: []
>>>
```

Explanation:

- n stores the number of elements, and an empty list arr is created.
- The first for loop takes n numbers from the user and adds them to arr.
- The second loop controls how many times the sorting process is repeated.
- The inner loop compares two neighboring elements: arr[j] and arr[j+1].
- If the first element is bigger, they are swapped using arr[j], arr[j+1] = arr[j+1], arr[j].
- Finally, print() displays the array in ascending order.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

B)Aim: Write a python programme to sort the elements using a insertion sort technique. Take an input elements from the users.

Programme:

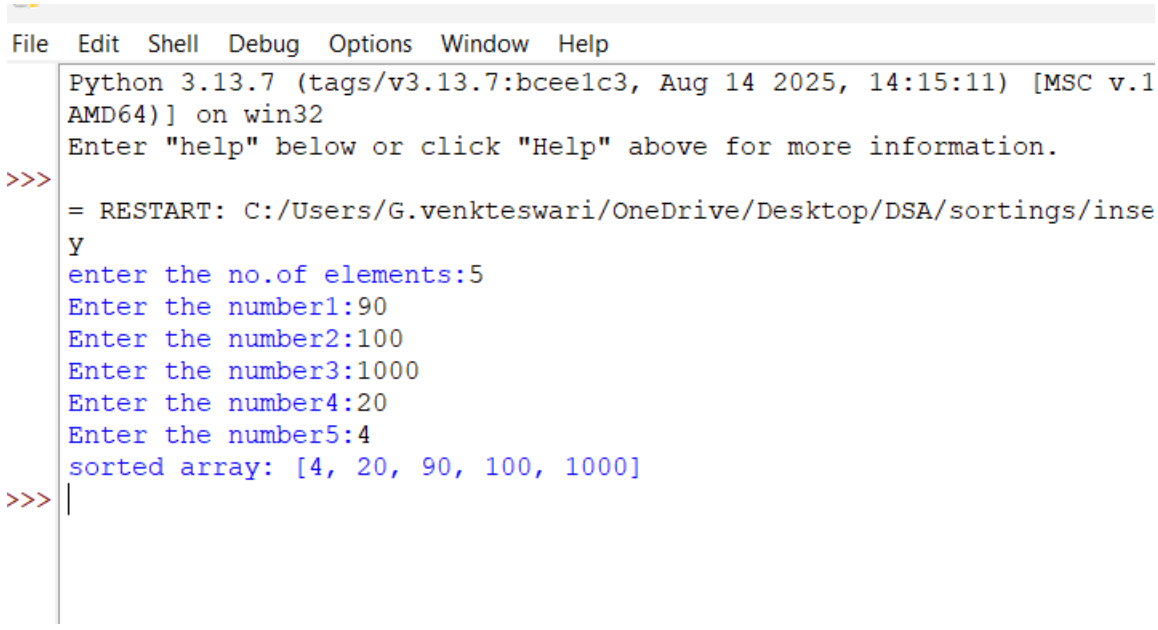
insertion sort.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/sortings/insert

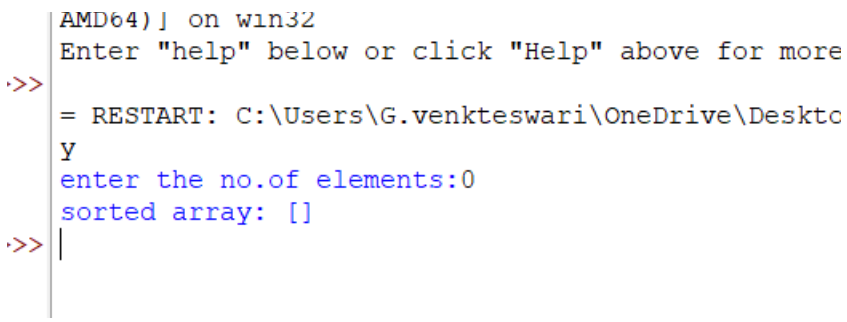
File Edit Format Run Options Window Help

```
#insertion sort

def insertion_sort(arr):
    n=len(arr)
    for j in range(1,n):
        key=arr[j]
        j=j-1
        while j>=0 and arr[j]>key:
            arr[j+1]=arr[j]
            j=j-1
        arr[j+1]=key
    return arr
n=int(input("enter the no.of elements:"))
arr=[]
for i in range(n):
    x=int(input(f"Enter the number{i+1}:"))
    arr.append(x)
insertion_sort(arr)
print("sorted array:",arr)
```

Output:





Explanation:

- insertion_sort(arr) takes the array and finds its length using n = len(arr).
- The for loop starts from the second element because the first element is considered already sorted.
- key = arr[j] stores the current element that we want to place in the correct position.
- The while loop moves bigger elements one position to the right.
- arr[j+1] = key places the current element in its correct position.
- Finally, the function returns the sorted array, and print() displays it in ascending order.

Error table:

S.no	Errors	Rectification

1.	Error in main logic	Rectified
2.	Errors in condition	rectified

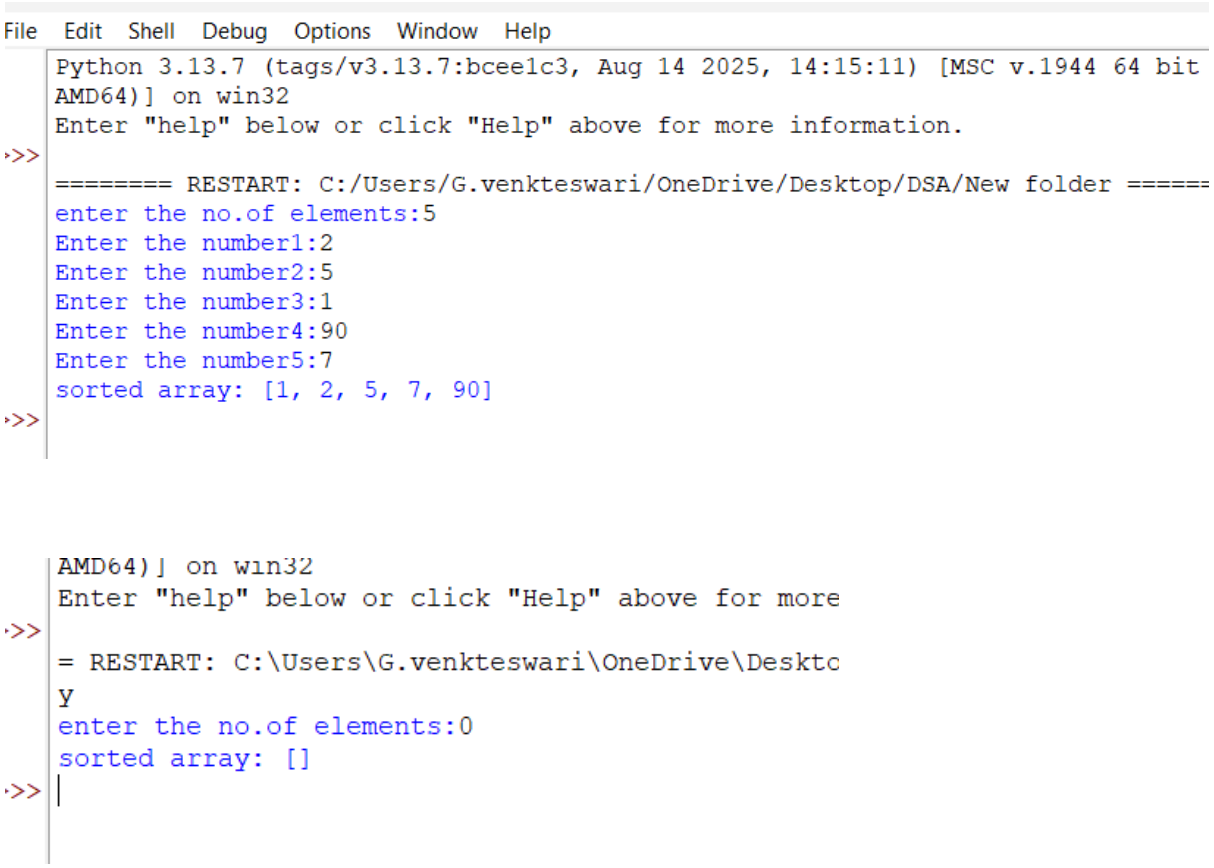
C)Aim: Write a python programme to sort the elements using a selection sort technique. Take an input elements from the users.

Programme:

File Edit Format Run Options Window Help

```
def selection_sort(arr):
    n=len(arr)
    for i in range(n-1):
        min_index=i
        for j in range(i+1,n):
            if arr[j]<arr[min_index]:
                min_index=j
        arr[i],arr[min_index]=arr[min_index],arr[i]
    return arr
n=int(input("enter the no.of elements:"))
arr=[]
for i in range(n):
    x=int(input(f"Enter the number{i+1}:"))
    arr.append(x)
selection_sort(arr)
print("sorted array:",arr)
```

Output:



Explanation:

- n = len(arr) finds the total number of elements in the array.
- The outer for loop selects each position where the smallest element should be placed.
- min_index = i assumes that the current position contains the smallest element.
- The inner for loop compares the remaining elements and finds the smallest element's index.
- arr[i], arr[min_index] = arr[min_index], arr[i] swaps the smallest element with the current element.
- After all iterations, the array is sorted in ascending order and print() displays it.

Error table:

S.no	Errors	Rectification
1.	Error in main logic	Rectified
2.	Errors in condition	rectified

D)Aim: Write a python programme to sort the elements using a merge sort technique. Take an input elements from the users.

Programme:

merge sort.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/sortings/merge sort.py (3.1...

File Edit Format Run Options Window Help

```
def merge_sort(arr):
    if len(arr)<=1:
        return arr
    mid=len(arr)//2
    left=arr[:mid]
    right=arr[mid:]
    merge_sort(left)
    merge_sort(right)
    i=j=k=0
    while i<len(left) and j<len(right):
        if left[i]<=right[j]:
            arr[k]=left[i]
            i=i+1
        else:
            arr[k]=right[j]
            j=j+1
        k=k+1
    while i<len(left):
        arr[k]=left[i]
        i=i+1
        k=k+1
    while j<len(right):
        arr[k]=right[j]
        j=j+1
        k=k+1
n=int(input("enter the no.of elements:"))
arr=[]
for i in range(n):
    x=int(input(f"Enter the number{i+1}:"))
    arr.append(x)
merge_sort(arr)
print("sorted array:",arr)
```

Output:

```
AMD64)] on win32
Enter "help" below or click "Help" above for more
>>>
= RESTART: C:\Users\G.venkteswari\OneDrive\Desktc
y
enter the no.of elements:0
sorted array: []
>>>
```

```
== RESTART: C:/Users/G.venkteswari/OneDrive/Desktop/DSA/sortings/merge sort.py =
enter the no.of elements:5
Enter the number1:7
Enter the number2:77
Enter the number3:8
Enter the number4:90
Enter the number5:100
sorted array: [7, 8, 77, 90, 100]
```

Explanation:

- if len(arr) <= 1 checks if the array has 0 or 1 element; if yes, it is already sorted.
- mid = len(arr)//2 finds the middle and divides the array into left and right parts.
- merge_sort(left) and merge_sort(right) recursively divide both parts into smaller parts.
- i = j = k = 0 creates indexes to compare elements from the left part, right part, and original array.
- The while loops compare elements and put the smaller element first into the original array.
- Finally, the remaining elements are added, and print() displays the sorted array in ascending order.

Error table:

S.no	Errors	Rectification
1.	Error in main logic	Rectified
2.	Errors in condition	rectified

E)Aim: Write a python programme to sort the elements using a merge sort technique. Take an input elements from the users.

Programme:

quick sort.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/sortings/quick sort.py (3.13.7)

File Edit Format Run Options Window Help

```
def quick_sort(a,low,high):
    if low<high:
        i=low
        j=high
        pivot=low
        while i<j:
            while i<len(a) and a[i]<=a[pivot]:
                i=i+1
            while a[j]>a[pivot]:
                j=j-1
            if i<j:
                a[i],a[j]=a[j],a[i]
        a[j],a[pivot]=a[pivot],a[j]
        quick_sort(a,low,j-1)
        quick_sort(a,j+1,high)
    return a
n=int(input("Enter the no.of elements:"))
a=[]
for i in range(n):
    x=int(input(f"enter the number{i+1}:"))
    a.append(x)
quick_sort(a,0,n-1)
print("sorted array:",a)
```

Output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>
== RESTART: C:/Users/G.venkteswari/OneDrive/Desktop/DSA/sortings/quick sort.py
Enter the no.of elements:5
enter the number1:1
enter the number2:3
enter the number3:2
enter the number4:0
enter the number5:1
sorted array: [0, 1, 1, 2, 3]
>
```

```
AMD64)] on win32
Enter "help" below or click "Help" above for more
>>>
= RESTART: C:\Users\G.venkteswari\OneDrive\Deskto
y
enter the no.of elements:0
sorted array: []
>>>
```

Explanation:

- quick_sort(a, low, high) sorts the elements between the low and high indexes.
- pivot = low selects the first element as the pivot.
- i moves from the left and finds an element greater than the pivot, while j moves from the right and finds an element smaller than or equal to the pivot.
- If i < j, a[i] and a[j] are swapped.
- After the loop, a[j] and the pivot are swapped, putting the pivot in its correct position.
- quick_sort() is called again for the left and right parts, and finally the array is printed in sorted order.

Error table:

S.no	Errors	Rectification
1.	Error in main logic	Rectified
2.	Errors in condition	rectified

Week-4

4. AIM: Write a python program to implement a single linked list which would perform the below operations, by taking an input from the user.

a. Create a linked list. b. Insert at beginning. c. Insert at end.

d. Insert at a specific index. e. Delete by value. f. Delete 1st node.
g. Delete last node. h. Count no. of nodes i. Display.

j. Exit.

Programme:

single linked list.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/single linked list.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SLL:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head
                while temp.next is not None:
                    temp = temp.next
                temp.next = new_node

        print("Linked List created successfully")

    # 2. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))

        new_node = Node(data)

        new_node.next = self.head
        self.head = new_node
        print(f"Inserted {data} at the beginning.")

    # 3. Insert at End
    def insert_end(self):
        data = int(input("Enter data: "))

        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return
```

single linked list.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/single linked list.py (3.13.7)

File Edit Format Run Options Window Help

```
        while temp.next is not None:
            temp = temp.next

        temp.next = new_node
        print(f"Inserted {data} at the end.")

    # 4. Insert at Index
    def insert_at_index(self):
        data = int(input("Enter data: "))
        index = int(input("Enter index: "))

        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            new_node = Node(data)
            new_node.next = self.head
            self.head = new_node
            return

        temp = self.head

        for i in range(index - 1):
            if temp is None:
                print("Index out of range")
                return
            temp = temp.next

        if temp is None:
            print("Index out of range")
            return

        new_node = Node(data)

        new_node.next = temp.next
        temp.next = new_node
        print(f"Inserted {data} at index {index}.")

    # 5. Delete from Beginning
    def delete_beginning(self):

        if self.head is None:
            print("List is empty")
            return

        self.head = self.head.next
```

single linked list.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/single linked list.py (3.13.7)

File Edit Format Run Options Window Help

```
        self.head = self.head.next
        print(f"Deleted beginning node.")

    # 6. Delete from End
    def delete_end(self):

        if self.head is None:
            print("List is empty")
            return

        if self.head.next is None:
            self.head = None
            return

        temp = self.head

        while temp.next.next is not None:
            temp = temp.next

        temp.next = None
        print(f"Deleted last node.")

    # 7. Delete from Index
    def delete_at_index(self):

        index = int(input("Enter index: "))

        if self.head is None:
            print("List is empty")
            return

        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            self.head = self.head.next
            return

        temp = self.head

        for i in range(index - 1):
            if temp.next is None:
                print("Index out of range")
                return
            temp = temp.next
```

single linked list.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/single linked list.py (3.13.7)

File Edit Format Run Options Window Help

```
    # 8. Count Nodes
    def count_nodes(self):

        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)
    # 9. Display
    def display(self):

        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")



# Main Program

sll = SLL()

while True:

    print("\n----- SINGLY LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete from Beginning")
    print("6. Delete from End")
    print("7. Delete from Index")
    print("8. Count Number of nodes")
    print("9. Display")
    print("10. Exit")
```

single linked list.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/single linked list.py (3.13.7)

File Edit Format Run Options Window Help

```
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete from Beginning")
    print("6. Delete from End")
    print("7. Delete from Index")
    print("8. Count Number of nodes")
    print("9. Display")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        sll.create()

    elif choice == 2:
        sll.insert_beginning()

    elif choice == 3:
        sll.insert_end()

    elif choice == 4:
        sll.insert_at_index()

    elif choice == 5:
        sll.delete_beginning()

    elif choice == 6:
        sll.delete_end()

    elif choice == 7:
        sll.delete_at_index()

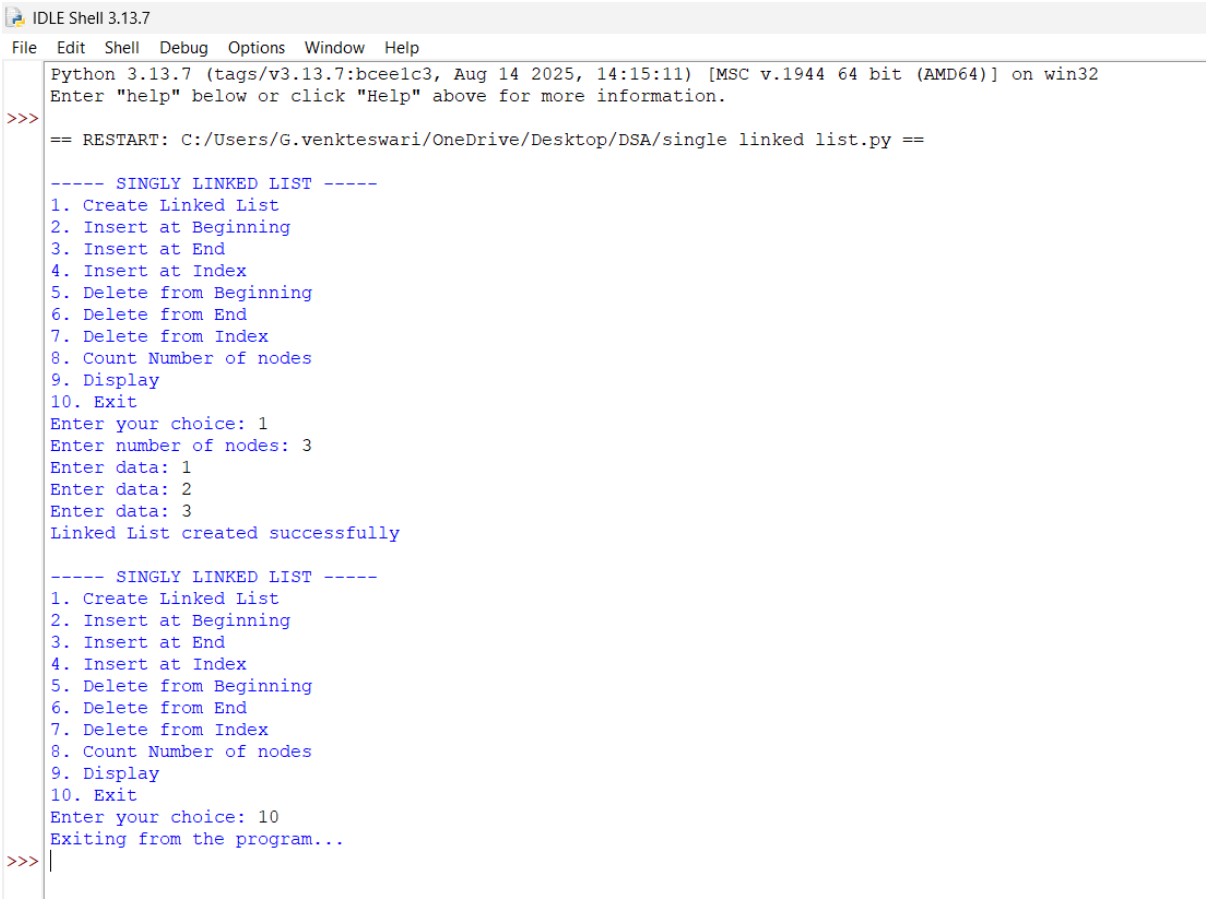
    elif choice == 8:
        sll.count_nodes()

    elif choice == 9:
        sll.display()

    elif choice == 10:
        print("Exiting from the program...")
        break

    else:
        print("Invalid choice")
```

Output:



Negative case:

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:\Users\G.venkteswari\OneDrive\Desktop\DSA\single linked list.py ==

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 17.8
Traceback (most recent call last):
  File "C:\Users\G.venkteswari\OneDrive\Desktop\DSA\single linked list.py", line
 204, in <module>
    sll.insert_beginning()
  File "C:\Users\G.venkteswari\OneDrive\Desktop\DSA\single linked list.py", line
 31, in insert_beginning
    data = int(input("Enter data: "))
ValueError: invalid literal for int() with base 10: '17.8'
>>>
```

Explanation:

1)Class Definitions:

1. Node class o Represents a single node in the linked list. o Each node has:

- data → stores the value.
- next → pointer to the next node (initially None).

2. SLL class o Manages the linked list operations. Has a head pointer (initially None) that points to the first node.

2)Insert at Beginning:

- Creates a new node.
- Points its next to current head.
- Updates head to new node.

3)Insert at End

• Creates a new node.

• If list is empty → new node becomes head.

• Otherwise → traverses to last node and attaches new node.

4) Insert at Index

• Asks for index and data.

• If index = 0 → behaves like "insert at beginning".

• Otherwise → traverses to (index-1) node and inserts new node in between.

5) Delete from Beginning

• Moves head to head.next.

• Effectively removes the first node.

6) Delete from End

• If list empty → prints "List is empty".

• If only one node → sets head = None.

• Otherwise → traverses to second-last node and sets its

next= None.

7) Delete from Index

• Asks for index.

• If index = 0 → removes head.

• Otherwise → traverses to (index-1) node and skips the next node.

8) Count Nodes

• Traverses through list.

• Counts nodes until None.

9) Display

• Traverses from head to end.

• Prints each node’s data in data -> data -> None format.

Error table:

sno	error	rectification
1.	Index out of rangeWhen inserting/deleting at an index greater than the number of nodes.	Traverse carefully,if temp becomes None before reaching index, print "Index out of range" and stop.
2.	Logical error in insertion if index is negative.	Validate index: if index < 0: print ("Invalid index").
3.	Indentation Error In the Count node part	It was rectified later, By maintaining proper indentation.

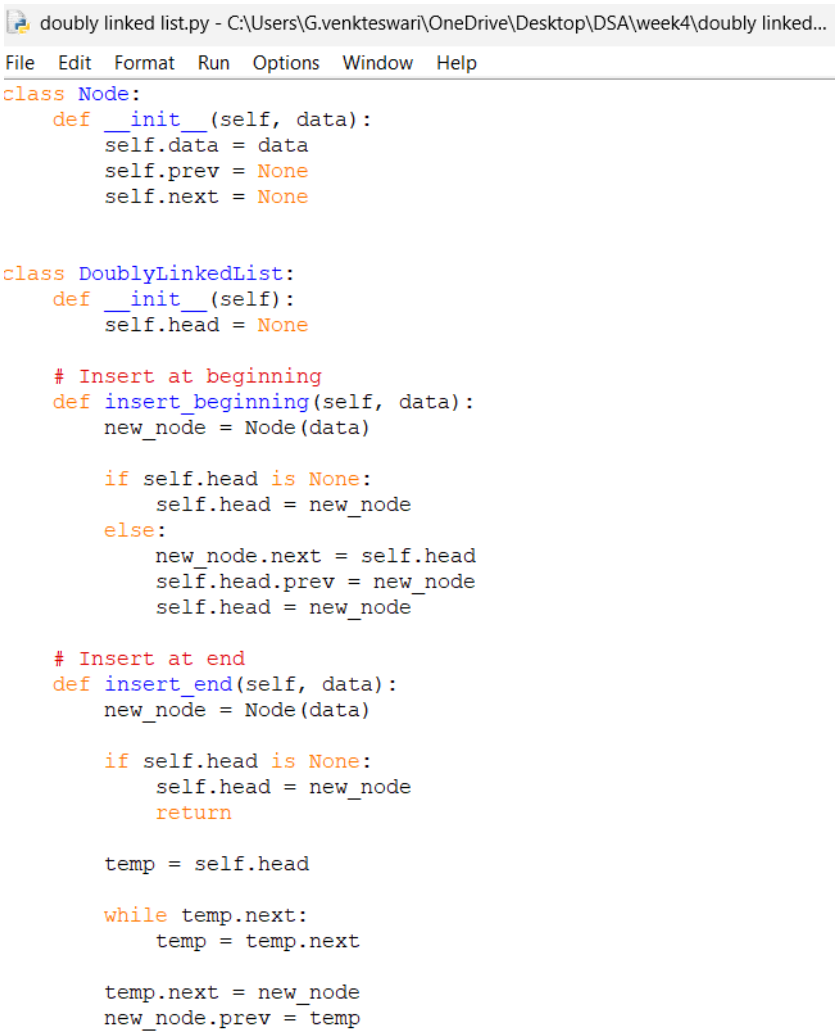
Aim: Write a python program to implement a doubly linked list which would perform the below operations, by taking an input from the user.

a. Create a linked list. b. Insert at beginning. c. Insert at end.

d. Insert at a specific index. e. Delete by value. f. Delete 1st node.
g. Delete last node. h. Count no. of nodes i. Display.

j. Exit.

Programme:



doubly linked list.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/week4/doubly linked list.py (3.13.7)

File Edit Format Run Options Window Help

```
    # Display backward
    def display_backward(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while temp.next:
            temp = temp.next

        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.prev

        print("None")


# Main program
dll = DoublyLinkedList()

while True:

    print("\n----- DOUBLY LINKED LIST -----")
    print("1. Insert at Beginning")
    print("2. Insert at End")
    print("3. Delete from Beginning")
    print("4. Delete from End")
    print("5. Display Forward")
    print("6. Display Backward")
    print("7. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        dll.insert_beginning(data)

    elif choice == 2:
        data = int(input("Enter data: "))
        dll.insert_end(data)

    elif choice == 3:
        dll.delete_beginning()

    elif choice == 4:
        dll.delete_end()
```

```
print("\n----- DOUBLY LINKED LIST -----")
print("1. Insert at Beginning")
print("2. Insert at End")
print("3. Delete from Beginning")
print("4. Delete from End")
print("5. Display Forward")
print("6. Display Backward")
print("7. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter data: "))
    dll.insert_beginning(data)

elif choice == 2:
    data = int(input("Enter data: "))
    dll.insert_end(data)

elif choice == 3:
    dll.delete_beginning()

elif choice == 4:
    dll.delete_end()

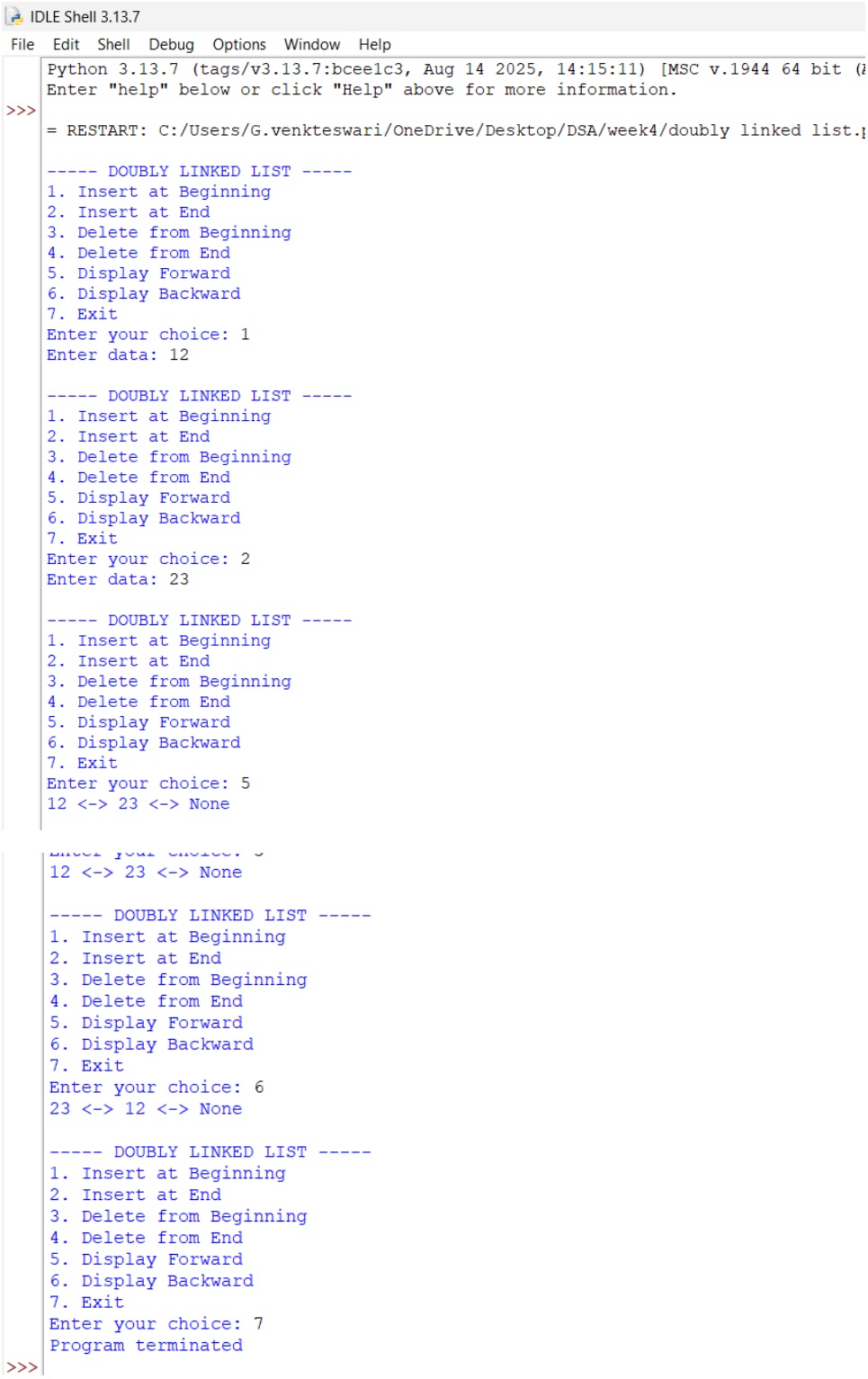
elif choice == 5:
    dll.display_forward()

elif choice == 6:
    dll.display_backward()

elif choice == 7:
    print("Program terminated")
    break

else:
    print("Invalid choice")
```

Output:



Explanation:

- Node has 3 parts:
  - data → stores value
  - prev → points to previous node

- next → points to next node

- Doubly Linked List:
  - Each node can move in both directions.
  - First node's prev is None.
  - Last node's next is None.
  - Example: None ← 10 ↔ 20 ↔ 30 → None

Operations

1. Insert at Beginning
   - Create a new node.
   - New node points to the current head.
   - Current head's prev points to new node.
   - New node becomes the head.
2. Insert at End
   - Traverse to the last node.
   - Last node's next points to new node.
   - New node's prev points to the last node.
3. Delete from Beginning
   - Move head to the next node.
   - Set new head's prev to None.
   - If only one node exists, list becomes empty.
4. Delete from End
   - Traverse to the last node.
   - Set the previous node's next to None.
   - If only one node exists, list becomes empty.
5. Display Forward

- Start from head.
- Move using next.
- Continue until None.

6. Display Backward
    - First move to the last node.
    - Move backward using prev.
    - Continue until None.
7. Main Program
    - Create a doubly linked list.
    - Display the menu.
    - Take user's choice.
    - Perform the selected operation.
    - Choice 7 exits the program.

Error table:

sno	error	rectification
1.	Index out of rangeWhen inserting/deleting at an index greater than the number of nodes.	Traverse carefully,if temp becomes None before reaching index, print "Index out of range" and stop.
2.	Logical error in insertion if index is negative.	Validate index: if index < 0: print ("Invalid index").
3.	Indentation Error In the Count node part	It was rectified later, By maintaining proper indentation.

Aim: Write a python program to implement a circular linked list which would perform the below operations, by taking an input from the user.

a. Create a linked list. b. Insert at beginning. c. Insert at end.

d. Insert at a specific index. e. Delete by value. f. Delete 1st node.
g. Delete last node. h. Count no. of nodes i. Display.

j. Exit.

Programme:

circular linked list.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/week4/circular linked list.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None

    # Insert at beginning
    def insert_beginning(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            new_node.next = self.head
            temp.next = new_node
            self.head = new_node

    # Insert at end
    def insert_end(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = new_node
            new_node.next = self.head
```

```
            temp.next = new_node
            new_node.next = self.head

        print("Node inserted at end.")

    # 4. Insert at specific index
    def insert_at_index(self):
        index = int(input("Enter index: "))
        data = int(input("Enter value: "))

        new_node = Node(data)

        if index == 0:
            if self.head is None:
                self.head = new_node
                new_node.next = self.head
            else:
                temp = self.head

                while temp.next != self.head:
                    temp = temp.next

                new_node.next = self.head
                temp.next = new_node
                self.head = new_node

            print("Node inserted successfully.")
            return

        if self.head is None:
            print("Invalid index.")
            return

        temp = self.head
        count = 0

        while count < index - 1:
            temp = temp.next
            count += 1
```

```
            temp = temp.next
            count += 1

            if temp == self.head:
                print("Invalid index.")
                return

        new_node.next = temp.next
        temp.next = new_node

        print("Node inserted successfully.")

    # 5. Delete by value
    def delete_by_value(self):
        value = int(input("Enter value to delete: "))

        if self.head is None:
            print("List is empty.")
            return

        current = self.head
        previous = None

        while True:
            if current.data == value:

                # Only one node
                if current == self.head and current.next == self.head:
                    self.head = None

                # Delete head
                elif current == self.head:
                    temp = self.head

                    while temp.next != self.head:
                        temp = temp.next

                    self.head = self.head.next
                    temp.next = self.head
```

circular linked list.py - C:\Users\G.venkteswari\OneDrive\Desktop\DSA\week4\circular linked list.py (3.13.7

File Edit Format Run Options Window Help

```
                    temp = self.head

                    while temp.next != self.head:
                        temp = temp.next

                    self.head = self.head.next
                    temp.next = self.head

                # Delete other node
                else:
                    previous.next = current.next

                print("Node deleted successfully.")
                return

            previous = current
            current = current.next

            if current == self.head:
                break

        print("Value not found.")

    # 6. Delete first node
    def delete_first(self):
        if self.head is None:
            print("List is empty.")
            return

        # Only one node
        if self.head.next == self.head:
            self.head = None
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            self.head = self.head.next
            temp.next = self.head

        print("First node deleted.")
```

circular linked list.py - C:\Users\G.venkteswari\OneDrive\Desktop\DSA\week4\circular linked list.py (3.13.7)

File Edit Format Run Options Window Help

```
        print("First node deleted.")

    # 7. Delete last node
    def delete_last(self):
        if self.head is None:
            print("List is empty.")
            return

        # Only one node
        if self.head.next == self.head:
            self.head = None
        else:
            temp = self.head

            while temp.next.next != self.head:
                temp = temp.next

            temp.next = self.head

        print("Last node deleted.")

    # 8. Count number of nodes
    def count_nodes(self):
        if self.head is None:
            print("Number of nodes: 0")
            return

        count = 1
        temp = self.head.next

        while temp != self.head:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)
```

circular linked list.py - C:\Users\G.venkteswari\OneDrive\Desktop\DSA\week4\circular linked list.py

File Edit Format Run Options Window Help

```
        print("Number of nodes:", count)

    # 9. Display
    def display(self):
        if self.head is None:
            print("List is empty.")
            return

        temp = self.head

        print("Circular Linked List:")

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next

            if temp == self.head:
                break

        print("(back to head)")


# Main program
cll = CircularLinkedList()

while True:

    print("\n========== CIRCULAR LINKED LIST ==========")
    print("1. Create a linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at a specific index")
    print("5. Delete by value")
    print("6. Delete 1st node")
    print("7. Delete last node")
    print("8. Count no. of nodes")
    print("9. Display")
    print("10. Exit")
```

```
    choice = int(input("Enter your choice: "))

    if choice == 1:
        cll.create()

    elif choice == 2:
        cll.insert_beginning()

    elif choice == 3:
        cll.insert_end()

    elif choice == 4:
        cll.insert_at_index()

    elif choice == 5:
        cll.delete_by_value()

    elif choice == 6:
        cll.delete_first()

    elif choice == 7:
        cll.delete_last()

    elif choice == 8:
        cll.count_nodes()

    elif choice == 9:
        cll.display()

    elif choice == 10:
        print("Exiting program...")
        break

    else:
        print("Invalid choice. Please try again.")
```

Output:



```
========== CIRCULAR LINKED LIST ==========
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at a specific index
5. Delete by value
6. Delete 1st node
7. Delete last node
8. Count no. of nodes
9. Display
10. Exit
Enter your choice: 9
Circular Linked List:
3 -> 3 -> 4 -> 1 -> 7 -> (back to head)

========== CIRCULAR LINKED LIST ==========
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at a specific index
5. Delete by value
6. Delete 1st node
7. Delete last node
8. Count no. of nodes
9. Display
10. Exit
Enter your choice: 10
Exiting program...
```

Explanation:

- Node contains two parts:
  - data → stores the value.
  - next → stores the reference of the next node.
- Circular Linked List:
  - Last node points back to the first node (head).
  - There is no None at the end.
  - Example: 10 → 20 → 30 → 10

Operations

1. Create
   - Creates n nodes.
   - First node becomes head.
   - Last node points to head.
2. Insert at Beginning
   - Create a new node.

- New node points to current head.
- Last node points to new node.
- New node becomes the new head.

3. Insert at End
    - Traverse until the last node.
    - Last node points to the new node.
    - New node points to head.
4. Insert at Specific Index
    - Move to the node before the required index.
    - Connect the new node between two nodes.
    - Newnode.next = temp.next
    - temp.next = new.node
5. Delete by Value
    - Search for the required value.
    - If found, remove that node.
    - Previous node is connected to the next node.
    - Special cases: only node or deleting the head.
6. Delete First Node
    - Move head to the second node.
    - Last node is connected to the new head.
7. Delete Last Node
    - Find the second-last node.
    - Make its next point to head.
8. Count Nodes
    - Start from head.

- Traverse until reaching head again.
- Count each node.

9. Display
    - Start from head.
    - Print each node.
    - Stop when head is reached again.

Error table:

sno	error	rectification
1.	Index out of rangeWhen inserting/deleting at an index greater than the number of nodes.	Traverse carefully,if temp becomes None before reaching index, print "Index out of range" and stop.
2.	Logical error in insertion if index is negative.	Validate index: if index < 0: print ("Invalid index").
3.	Indentation Error In the Count node part	It was rectified later, By maintaining proper indentation.

WEEK-5

AIM: Write a programe of stack using array

Programme:

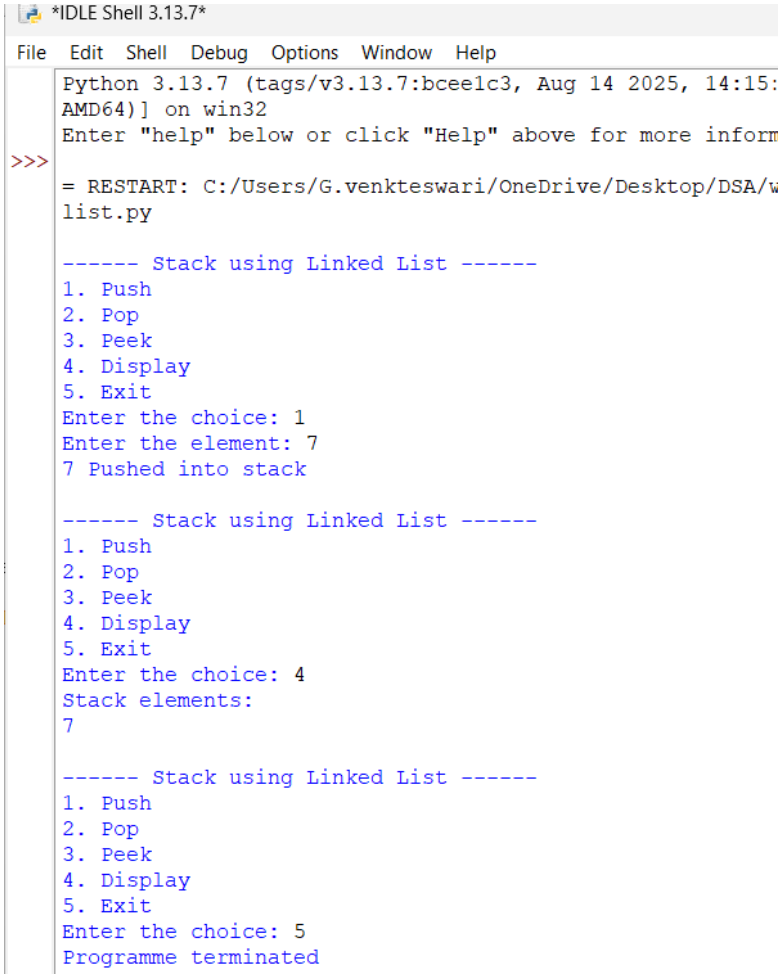
stack using array.py - C:\Users\G.venkteswari\OneDrive\Desktop\DSA\week5\stack using array.py (

File Edit Format Run Options Window Help

```
#stack using array:
class StackEx:
    def __init__(self, size):
        self.size=size
        self.stack=[None]*size
        self.top = -1
    def push(self,item):
        if self.top == self.size - 1:
            print("stack overflow")
        else:
            self.top+=1
            self.stack[self.top]=item
            print(item,"pushed into the stack")
    def pop(self):
        if self.top == -1:
            print("stack underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top]=None
            self.top-=1
            print(item,"popped from the stack")
    def peek(self):
        if self.top == -1:
            print("stack is empty")
        else:
            print("Top Element:",self.stack[self.top])
    def display(self):
        if self.top == -1:
            print("stack is empty")
        else:
            print("The elements of the stack are:")
            for i in range(self.top,-1,-1):
                print(self.stack[i])
size=int(input("enter the size of the stack:"))
s = StackEx(size)
```

```
while True:
    print("\n---------Stack Menu---------\n")
    print("1.push\n")
    print("2.pop\n")
    print("3.peek\n")
    print("4.display")
    choice = int(input("Enter the choice(1 to 5):"))
    if choice == 1:
        item = int(input("enter the element to push:"))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Programme terminated")
        break
    else:
        print("Invalid choice")
```

Output:



Explanation:

- Stack is a linear data structure.it follows the LIFO (last in first out).
- Elements are stored in the array
- Push is used to insert an element. The element is added at the top. If the stack is full, Stack Overflow occurs.
- Pop is used to remove an element.
- The top element is removed.
- top is decreased. If the stack is empty, Stack Underflow occurs.
- Peek is used to view the top element.
- It does not remove the element.
- Display is used to show all elements.
- Elements are displayed from top to bottom.

Error table:

Error	Rectified
Forget to keep (_ _) in constructor	Rectified by keep double underscore.
Forgot to decrease the top in pop	rectified

AIM: Stack using Linked list.

Programme:

stack using linked list.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/week5/stack usi

File Edit Format Run Options Window Help

```
#stack using linked list:
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        new = Node(data)
        new.next = self.top
        self.top = new
        print(data, "Pushed into stack")

    def pop(self):
        if self.top is None:
            print("Stack underflow")
        else:
            temp = self.top
            print(temp.data, "popped from stack")
            self.top = self.top.next

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
            print("Stack elements:")
            while temp is not None:
                print(temp.data)
                temp = temp.next


s = Stack()
```

```
while True:
    print("\n------ Stack using Linked List ------")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter the choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        s.push(data)

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        print("Programme terminated")
        break

    else:
        print("Invalid choice")
```

Output:

```
File  Edit  Shell  Debug  Options  Window  Help
    Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11
    AMD64)] on win32
    Enter "help" below or click "Help" above for more informat
>>>
    = RESTART: C:/Users/G.venkteswari/OneDrive/Desktop/DSA/wee
    list.py

    ------ Stack using Linked List ------
    1. Push
    2. Pop
    3. Peek
    4. Display
    5. Exit
    Enter the choice: 1
    Enter the element: 6
    6 Pushed into stack

    ------ Stack using Linked List ------
    1. Push
    2. Pop
    3. Peek
    4. Display
    5. Exit
    Enter the choice: 4
    Stack elements:
    6

    ------ Stack using Linked List ------
    1. Push
    2. Pop
    3. Peek
    4. Display
    5. Exit
    Enter the choice: 5
    Programme terminated
>>>
```

Explanation:

- A stack using linked list is a linear data structure that follows the LIFO (Last In, First Out) principle.
- In a linked-list stack, each element is stored in a node, and each node contains data and a link to the next node.
- A variable called top is used to point to the first node, which represents the top of the stack.
- Push operation is used to insert a new element. A new node is created and added at the top of the stack.
  Pop operation is used to remove an element. The node pointed to by top is removed and top moves to the next node.
- Peek operation is used to view the top element without removing it from the stack.
- Display operation is used to traverse the linked list from top to None and display all the elements.

- A linked-list stack has dynamic size, so memory is allocated as needed. Stack Underflow occurs when we try to remove an element from an empty stack.

Error Table:

Error	Rectified
new. next = None	new. next = self.top
self. top = self.top	self. top = self. top. next

WEEK-6

AIM: Write a python programme to implement queue and its operations such as inqueue, dequeue, peek and display using array.

Programme:

queue with array.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/week6/

File Edit Format Run Options Window Help

```
class Queue:
    def __init__(self, size):

        self.size = size

        self.queue = [None] * size

        self.front = -1

        self.rear = -1

    def enqueue(self, x):

        if self.rear == self.size - 1:

            print("Queue overflow")

        else:

            if self.front == -1:

                self.front = 0

            self.rear += 1

            self.queue[self.rear] = x

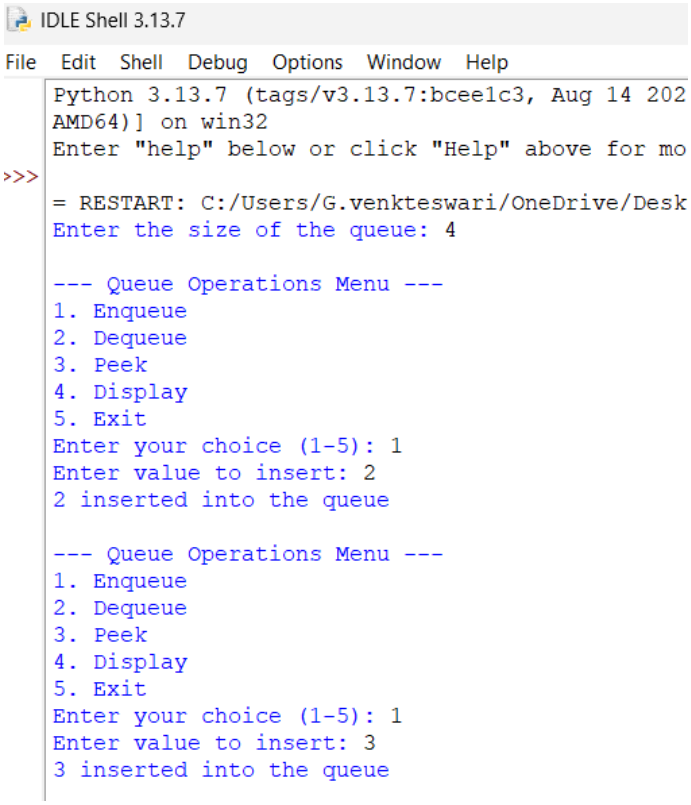
            print(f"{x} inserted into the queue")
```

```
    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("queue underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            self.front += 1
            print(f"{x} deleted from the queue")
            if self.front > self.rear:
                self.front = -1
                self.rear = -1
    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front elements:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("the elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])
if __name__ == "__main__":
    size = int(input("Enter the size of the queue: "))
    q = Queue(size)
    while True:
        print("\n--- Queue Operations Menu ---")
        print("1. Enqueue")
        print("2. Dequeue")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")
        choice = input("Enter your choice (1-5): ")
```

```
        if choice == '1':
            val = int(input("Enter value to insert: "))
            q.enqueue(val)
        elif choice == '2':
            q.dequeue()
        elif choice == '3':
            q.peek()
        elif choice == '4':
            q.display()
        elif choice == '5':
            print("Exiting program.")
            break
        else:
            print("Invalid choice! Please enter a number between 1 and 5.")
```

Output:



```
--- Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 4
the elements of the queue are:
2
3

--- Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 5
Exiting program.
```

Explanation:

- Queue is the linear data structure. It follows first in first out rule.
- In queue for insertion we enqueue and for deletion we use dequeue.
- Before inserting an element we check whether the queue is filled or if it is filled then it is queue overflow.
- Before deletion we will check whether any element is present in the queue or not if any element is not present then it is queue underflow.
- For seeking the top most element we use peek.
- And for inserting, deleting, peek and for display we use menu driven system.
- Users choice will be considerable.

Error table:

S.no	Errors	Rectified
1.	Forgetting the placing of semicolons	Rectified
2.	Indentation errors	Rectified

AIM: Write a python programme to implement queue and its operations such as enqueue, dequeue, peek and display using Linked list data structure.

Programme:

queue with list.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/v

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
    def enqueue(self, x):
        new_node = Node(x)
        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node
        print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front is None:
            print("Queue underflow (Queue is empty)")
        else:
            x = self.front.data
            self.front = self.front.next
            if self.front is None:
                self.rear = None
            print(f"{x} deleted from the queue")
    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)
```

```
    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            current = self.front
            while current:
                print(current.data, end=" -> " if current.next else "")
                current = current.next
            print()

if __name__ == "__main__":
    q = Queue()
    while True:
        print("\n--- Queue (Linked List) Operations Menu ---")
        print("1. Enqueue")
        print("2. Dequeue")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")
        choice = input("Enter your choice (1-5): ")
        if choice == '1':
            val = int(input("Enter value to insert: "))
            q.enqueue(val)
    elif choice == '2':
        q.dequeue()
    elif choice == '3':
        q.peek()
    elif choice == '4':
        q.display()
    elif choice == '5':
        print("Exiting program.")
        break
    else:
        print("Invalid choice! Please enter a number between 1 and 5.")
```

Output:

```
>>
= RESTART: C:/Users/G.venkteswari/OneDrive/Deskto

--- Queue (Linked List) Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 1
Enter value to insert: 7
7 inserted into the queue

--- Queue (Linked List) Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 1
Enter value to insert: 4
4 inserted into the queue

--- Queue (Linked List) Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 4
The elements of the queue are:
7 -> 4


--- Queue (Linked List) Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 5
Exiting program.
```

Explanation:

- Each node stores data and a link (next) to the next node.
- Has two pointers: front (start of queue) and rear (end of queue).
- Creates a new node.
- If queue is empty → both front and rear point to this node.
- Otherwise → attaches new node at the end and moves rear.
- Removes the node at front.

- Moves front to the next node.
- If queue becomes empty → rear also set to None.
- Shows the data at front without removing it.
- If empty → prints *Queue is empty*.
- Starts from front and prints each node until the end.
- Shows elements in order with arrows (->).
- Enqueue works fine on empty queue.
- Dequeue, Peek, Display all check if queue is empty before acting.
- Unlike array-based queue, linked list queue grows/shrinks as needed.
- No fixed size or overflow problem (only limited by memory).
- User chooses operations: Enqueue, Dequeue, Peek, Display, Exit.
- Runs in a loop until Exit is chosen.

Error table:

S.no	Errors	Rectification
1.	Indentation errors	Rectified
2.	Forgetting of placing semi colon.	Rectified

AIM: Write a python programme to implement queue and its operations such as enqueue, dequeue, peek and display using Linked list data structure.

Programme:

File Edit Format Run Options Window Help

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1
    def enqueue(self, x):
        if (self.rear + 1) % self.size == self.front:
            print("Queue overflow")
        else:
            if self.front == -1:
                self.front = 0
                self.rear = 0
            else:
                self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")

   def dequeue(self):
        if self.front == -1:
            print("Queue underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            if self.front == self.rear:
                self.front = -1
                self.rear = -1
            else:
                self.front = (self.front + 1) % self.size
            print(f"{x} deleted from the queue")
```

```
    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            i = self.front
            while True:
                print(self.queue[i], end=" ")
                if i == self.rear:
                    break
                i = (i + 1) % self.size
            print()

if __name__ == "__main__":
    size = int(input("Enter the size of the circular queue: "))
    q = CircularQueue(size)
    while True:
        print("\n--- Circular Queue Operations Menu ---")
        print("1. Enqueue")
        print("2. Dequeue")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")
        choice = input("Enter your choice (1-5): ")
```

```
    if choice == '1':
        val = int(input("Enter value to insert: "))
        q.enqueue(val)
    elif choice == '2':
        q.dequeue()
    elif choice == '3':
        q.peek()
    elif choice == '4':
        q.display()
    elif choice == '5':
        print("Exiting program.")
        break
    else:
        print("Invalid choice! Please enter a number between 1 and 5.")
```

Output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:1
AMD64)] on win32
Enter "help" below or click "Help" above for more informa

= RESTART: C:/Users/G.venkteswari/OneDrive/Desktop/DSA/we
Enter the size of the circular queue: 4

--- Circular Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 1
Enter value to insert: 7
7 inserted into the queue

--- Circular Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 3
Front element: 7

--- Circular Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 4
The elements of the queue are:
7
```

```
--- Circular Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 5
Exiting program.
```

Explanation:

- A circular queue uses a fixed-size list but connects the end back to the start, so space is reused.
- front and rear keep track of where elements are removed and added.
- When inserting, if (rear + 1) % size == front, the queue is full → overflow.
- If the queue is empty, both front and rear are set to 0 before inserting the first element.
- On enqueue, rear moves forward in a circular way using (rear + 1) % size.
- On dequeue, the element at front is removed, and front moves forward in a circular way.
- If after removing, front == rear, the queue becomes empty again (front = rear = -1).
- Display prints elements starting from front and looping until rear, showing the circular order.

Error table:

S.no	Errors	Rectified
1.	Indentation error	Rectified
2.	Forgetting to place front==rear	Rectified

WEEK-7

Aim: Write a python programme on binary tree using list and implement inorder, postorder, preorder, levelorder.

Programme:

binary tree using list.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self,data):
        self.data=data
        self.left=None
        self.right=None

class BinaryTree:
    def __init__(self):
        self.root=None

    def create_tree(self):
        self.root=Node('A')
        self.root.left=Node('B')
        self.root.right=Node('C')
        self.root.left.left=Node('D')
        self.root.left.right=Node('E')
        self.root.right.right=Node('F')

    def preorder(self,node):
        if node is None:
            return
        print(node.data,end=" ")
        self.preorder(node.left)
        self.preorder(node.right)

    def inorder(self,node):
        if node is None:
            return
        self.inorder(node.left)
        print(node.data,end=" ")
        self.inorder(node.right)
```

```
    def postorder(self,node):
        if node is None:
            return
        self.postorder(node.left)
        self.postorder(node.right)
        print(node.data,end=" ")

    def level_order(self):
        if self.root is None:
            return
        queue=[self.root]
        while queue:
            node=queue.pop(0)
            print(node.data,end=" ")
            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)

tree=BinaryTree()
tree.create_tree()

print("Preorder: ",end="")
tree.preorder(tree.root)

print("\nInorder: ",end="")
tree.inorder(tree.root)

print("\nPostorder: ",end="")
tree.postorder(tree.root)

print("\nLevel-order: ",end="")
tree.level_order()
```

Output:

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11)
AMD64)] on win32
Enter "help" below or click "Help" above for more informatio
>>>
= RESTART: C:/Users/G.venkteswari/OneDrive/Desktop/DSA/week7
ist.py
Preorder: A B D E C F
Inorder: D B E A C F
Postorder: D E B F C A
Level-order: A B C D E F
>>>
```

Explanation:

- The Node class defines each tree element with data, left, and right links.
- The BinaryTree class manages the tree, starting with root = None.

- create_tree() builds a fixed binary tree with nodes A–F arranged in a specific structure.
- Recursive methods (preorder, inorder, postorder) traverse the tree in different orders and print node values.
- level_order() uses a queue to perform breadth-first traversal, visiting nodes level by level.
- The program prints traversal outputs: Preorder → A B D E C F, Inorder → D B E A C F, Postorder → D E B F C A, Level-order → A B C D E F.
- The recursive traversal functions stop when they reach a None node, ensuring the recursion terminates correctly.
- The queue in level_order() is a simple list, and nodes are processed in FIFO order to achieve breadth-first traversal.
- The print(node.data,end=" ") statements ensure all outputs appear on the same line with spaces instead of newlines.
- The tree structure is hardcoded in create_tree(); it doesn't take user input, so the same tree is always built.
- Each traversal method demonstrates a different way of exploring the same tree, useful for understanding tree algorithms.
- The program's final output shows how traversal orders differ, helping visualize the relationship between tree structure and traversal logic.

Error Table:

S.no	Errors	Rectified
1.	Instead of writing node written nod	Rectified
2.	Syntax error that is (:)	Rectified

Aim: Write a python programme on binary tree using array and implement inorder, preorder, postorder and levelorder.

Programme:

binary tree using array.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/week7/binary tree using array.

File Edit Format Run Options Window Help

```
class BinaryTreeArray:
    def __init__(self,size):
        self.tree=[None]*size
        self.size=size
    def set_root(self,data):
        self.tree[0]=data
    def set_left(self,parent_index,data):
        child_index=2*parent_index+1
        if child_index<self.size:
            self.tree[child_index]=data
        else:
            print("Index out of range!")
    def set_right(self,parent_index,data):
        child_index=2*parent_index+2
        if child_index<self.size:
            self.tree[child_index]=data
        else:
            print("Index out of range!")
    def preorder(self,index=0):
        if index>=self.size or self.tree[index] is None:
            return
        print(self.tree[index],end=" ")
        self.preorder(2*index+1)
        self.preorder(2*index+2)
    def inorder(self,index=0):
        if index>=self.size or self.tree[index] is None:
            return
        self.inorder(2*index+1)
        print(self.tree[index],end=" ")
        self.inorder(2*index+2)
    def postorder(self,index=0):
        if index>=self.size or self.tree[index] is None:
            return
        self.postorder(2*index+1)
        self.postorder(2*index+2)
        print(self.tree[index],end=" ")
```

```
        print(self.tree[index],end=" ")
    def level_order(self):
        for value in self.tree:
            if value is not None:
                print(value,end=" ")
    def display(self):
        print("\nArray Representation:")
        for i in range(self.size):
            print(f"Index {i}: {self.tree[i]}")
tree=BinaryTreeArray(7)
tree.set_root('A')
tree.set_left(0,'B')
tree.set_right(0,'C')
tree.set_left(1,'D')
tree.set_right(1,'E')
tree.set_right(2,'F')
tree.display()
print("\nPreorder: ",end="")
tree.preorder()
print("\nInorder: ",end="")
tree.inorder()
print("\nPostorder: ",end="")
tree.postorder()
print("\nLevel-order: ",end="")
tree.level_order()
```

Output:

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:1
AMD64)] on win32
Enter "help" below or click "Help" above for more infc
>>>
= RESTART: C:/Users/G.venkteswari/OneDrive/Desktop/DSA
rray.py

Array Representation:
Index 0: A
Index 1: B
Index 2: C
Index 3: D
Index 4: E
Index 5: None
Index 6: F

Preorder: A B D E C F
Inorder: D B E A C F
Postorder: D E B F C A
Level-order: A B C D E F
>>>
```

Explanation:

- The code creates a binary tree using an array instead of linked nodes.
- The root is stored at index 0, and children are calculated using formulas: left child → 2*parent_index+1, right child → 2*parent_index+2.
- If the calculated index is within the array size, the child is added; otherwise, it prints "Index out of range!".
- Preorder traversal prints the current node first, then left child, then right child.
- Inorder traversal prints the left child, then the current node, then the right child.
- Postorder traversal prints children first, then the current node.
- Level order traversal simply loops through the array and prints all non-None values.
- The display() method shows the array representation with index numbers and stored values.
- The tree is built with nodes A–F, and then all traversals are executed to show different ways of visiting nodes.
- This demonstrates how binary trees can be represented and traversed using arrays instead of node pointers.

Errortable:

S.no	Errors	Rectified
1.	Printing none values in level order	Rectified
2.	Root is not set before traversal	Rectified by calling set_root() first.

WEEK-8

Aim: Write a python programme to implement bineary search tree with its operation.

Program:

binary search tree.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/week8/binary search t

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, data):
    if root is None:
        return Node(data)
    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)
    return root

def search(root, key):
    if root is None or root.data == key:
        return root
    if key < root.data:
        return search(root.left, key)
    return search(root.right, key)

def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)

def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

def delete(root, key):
    if root is None:
        return root
```

binary search tree.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/week8/binary

File Edit Format Run Options Window Help

```
    if key < root.data:
        root.left = delete(root.left, key)
    elif key > root.data:
        root.right = delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left

        temp = root.right
        while temp.left:
            temp = temp.left

        root.data = temp.data
        root.right = delete(root.right, temp.data)

    return root

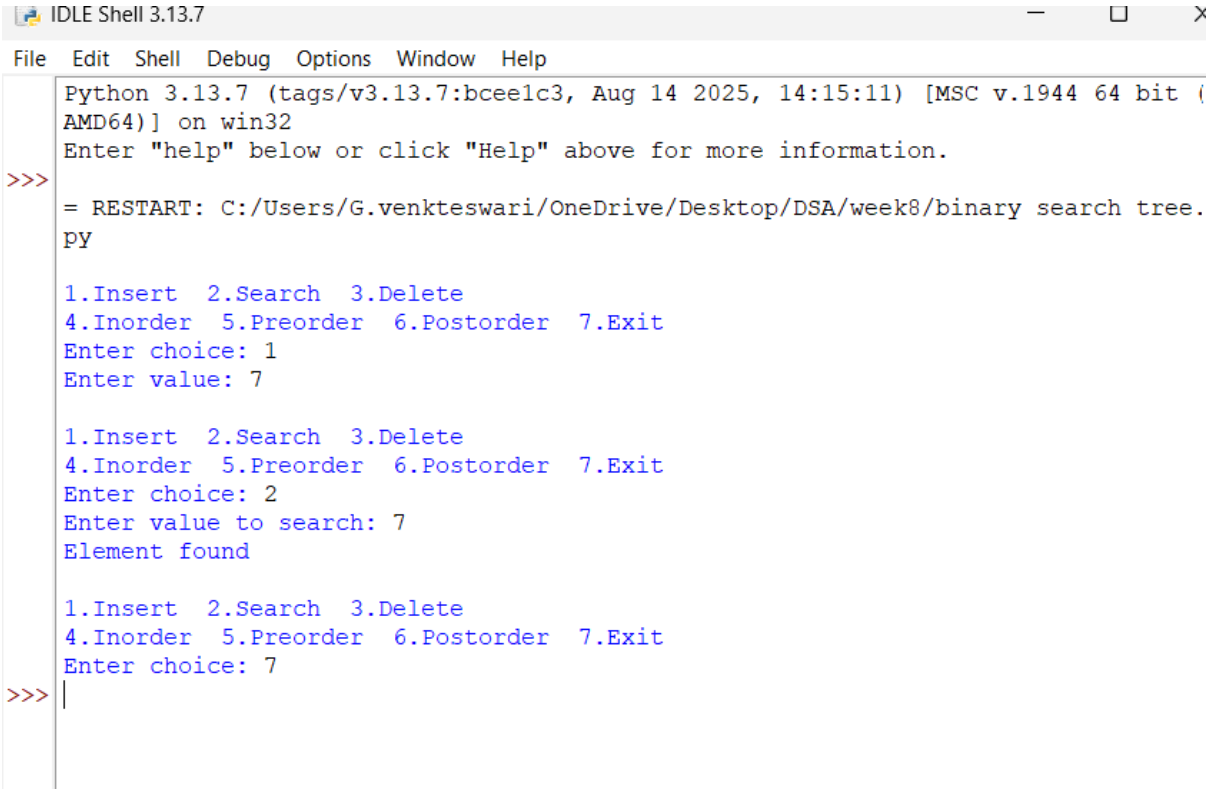
root = None

while True:
    print("\n1.Insert  2.Search  3.Delete")
    print("4.Inorder  5.Preorder  6.Postorder  7.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        x = int(input("Enter value: "))
        root = insert(root, x)
    elif ch == 2:
        x = int(input("Enter value to search: "))
        if search(root, x):
            print("Element found")
        else:
            print("Element not found")

            print("Element not found")
    elif ch == 3:
        x = int(input("Enter value to delete: "))
        root = delete(root, x)
    elif ch == 4:
        inorder(root)
        print()
    elif ch == 5:
        preorder(root)
        print()
    elif ch == 6:
        postorder(root)
        print()
    elif ch == 7:
        break
    else:
        print("Invalid choice")
```

Output:



Explanation:

1. The Node class is used to create a node containing data and two pointers, left and right, which point to the left and right child nodes.

2. The insert() function adds a new node to the Binary Search Tree. Smaller values are placed on the left side, and larger values are placed on the right side.

3. The search() function checks whether the given element exists in the tree. It compares the element with the current node and searches the left or right subtree.

4. The delete() function removes a node from the tree. It handles nodes with no children, one child, or two children while maintaining the BST property.

5. The inorder(), preorder(), and postorder() functions display the tree elements in different orders. Inorder displays elements in ascending order.

6. The while loop displays a menu and allows the user to choose operations such as insert, search, delete, and traversal. The program continues until the user selects Exit.

Error table:

S.no	Errors	Rectified
1.	Using an undefined variable or function name	Rectified
2.	Passing a value of the wrong data type.	Rectified.

WEEK-9

Aim: Write a python program to implement avl trees with its operation.

Program:

avl tree.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/wkke9/avl tree.py (3.13.7)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1

def height(root):
    if root is None:
        return 0
    return root.height

def balance(root):
    if root is None:
        return 0
    return height(root.left) - height(root.right)

def rightRotate(y):
    x = y.left
    t = x.right

    x.right = y
    y.left = t

    y.height = 1 + max(height(y.left), height(y.right))
    x.height = 1 + max(height(x.left), height(x.right))
    return x

def leftRotate(x):
    y = x.right
    t = y.left

    y.left = x
    x.right = t

    x.height = 1 + max(height(x.left), height(x.right))
    y.height = 1 + max(height(y.left), height(y.right))
    return y
```

avl tree.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/wkke9/avl tree.py (3.13.7)

File Edit Format Run Options Window Help

```
def insert(root, data):
    if root is None:
        return Node(data)

    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)
    else:
        return root

    root.height = 1 + max(height(root.left), height(root.right))
    b = balance(root)

    if b > 1 and data < root.left.data:
        return rightRotate(root)

    if b < -1 and data > root.right.data:
        return leftRotate(root)

    if b > 1 and data > root.left.data:
        root.left = leftRotate(root.left)
        return rightRotate(root)

    if b < -1 and data < root.right.data:
        root.right = rightRotate(root.right)
        return leftRotate(root)

    return root

def search(root, key):
    if root is None or root.data == key:
        return root
    if key < root.data:
        return search(root.left, key)
    return search(root.right, key)

def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)
```

avl tree.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/wkke9/avl tree.py (3.13.7)

File Edit Format Run Options Window Help

```
        print(root.data, end=" ")
        inorder(root.right)

def minNode(root):
    while root.left:
        root = root.left
    return root

def delete(root, key):
    if root is None:
        return root

    if key < root.data:
        root.left = delete(root.left, key)
    elif key > root.data:
        root.right = delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left

        temp = minNode(root.right)
        root.data = temp.data
        root.right = delete(root.right, temp.data)

    if root is None:
        return root

    root.height = 1 + max(height(root.left), height(root.right))
    b = balance(root)

    if b > 1 and balance(root.left) >= 0:
        return rightRotate(root)

    if b > 1 and balance(root.left) < 0:
        root.left = leftRotate(root.left)
        return rightRotate(root)

    if b < -1 and balance(root.right) <= 0:
        return leftRotate(root)

    if b < -1 and balance(root.right) > 0:
        root.right = rightRotate(root.right)
        return leftRotate(root)
```

```
        return leftRotate(root)

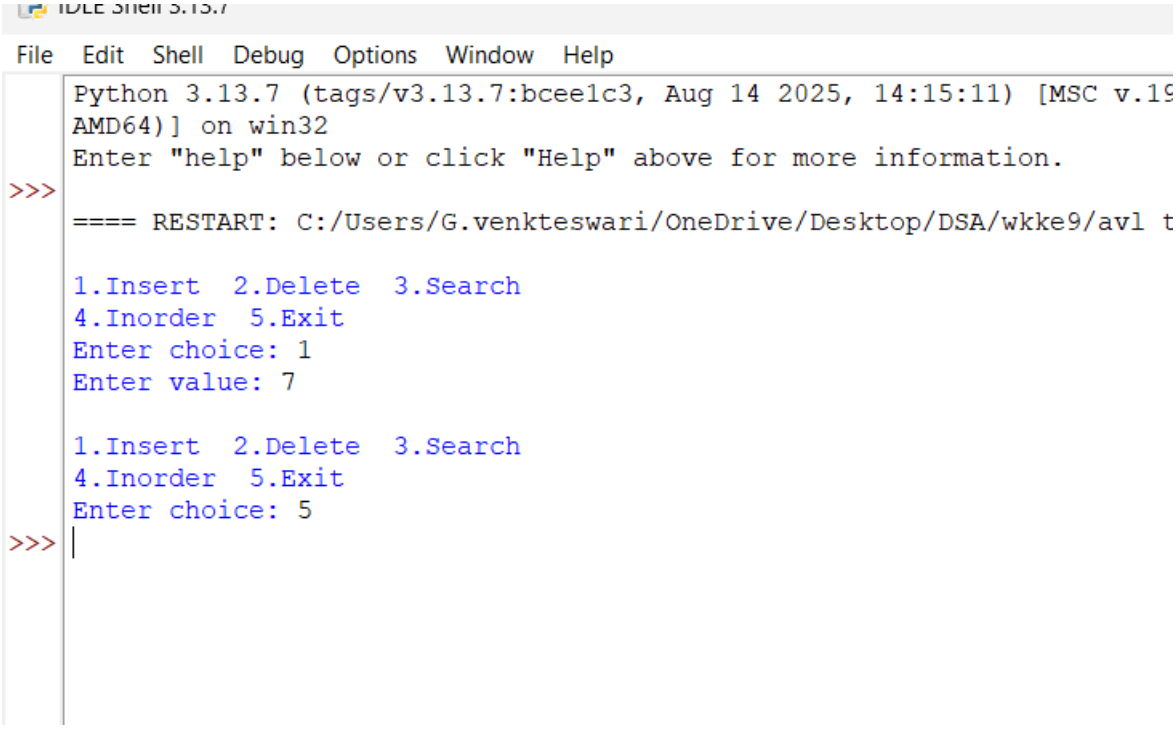
    return root

root = None

while True:
    print("\n1.Insert  2.Delete  3.Search")
    print("4.Inorder  5.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        x = int(input("Enter value: "))
        root = insert(root, x)
    elif ch == 2:
        x = int(input("Enter value to delete: "))
        root = delete(root, x)
    elif ch == 3:
        x = int(input("Enter value to search: "))
        if search(root, x):
            print("Element found")
        else:
            print("Element not found")
    elif ch == 4:
        inorder(root)
        print()
    elif ch == 5:
        break
    else:
        print("Invalid choice")
```

Output:



Explanation:

- The Node class creates a node with data, left and right pointers, and height.
- The height() and balance() functions calculate the height and balance factor of each node.
- The leftRotate() and rightRotate() functions balance the tree when it becomes unbalanced.
- The insert() function inserts a new element and performs rotations when necessary to maintain AVL balance.
- The delete() function removes an element and balances the tree again. The search() function checks whether an element exists.
- The inorder() function displays the elements in ascending order, while the menu allows the user to select different operations.

Error table:

S.no	Errors	Rectified
1.	Using an undefined variable or function.	Rectified
2.	Entering a non-integer value for the menu choice.	Rectified.

WEEK-10

Aim: Python programs for hashing using separate chaining, linear probing, and quadratic probing.

Program:

hashing using seperate chaining.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/

File Edit Format Run Options Window Help

```
size = 10
table = [[] for i in range(size)]

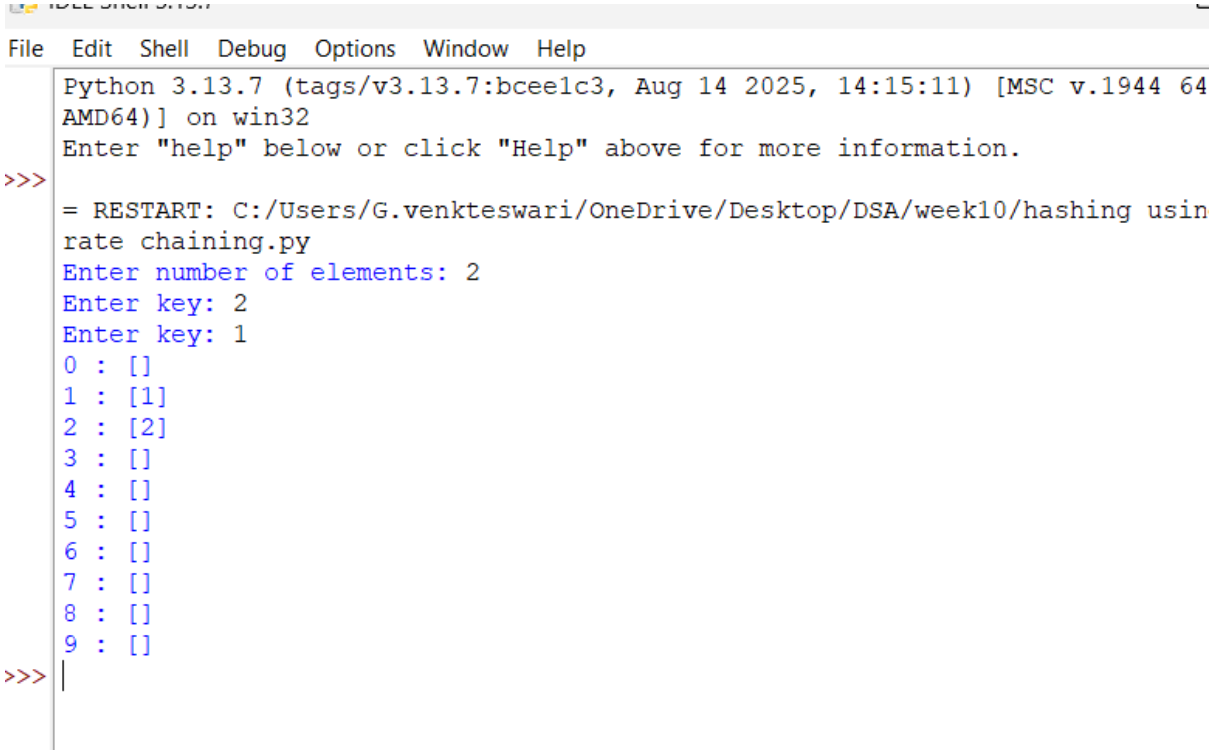
def insert(key):
    index = key % size
    table[index].append(key)

def display():
    for i in range(size):
        print(i, ":", table[i])

n = int(input("Enter number of elements: "))
for i in range(n):
    key = int(input("Enter key: "))
    insert(key)

display()
```

Output:



Explanation:

- The size = 10 statement creates a hash table with 10 positions.
- The table = [[] for i in range(size)] statement creates an empty list at every position.
- The insert() function calculates the index using key % size.
- If two keys have the same index, both keys are stored in the same list.
- The display() function prints each index and its stored elements.
- The for loop accepts keys from the user and inserts them into the hash table.

Error table:

S.no	Errors	Rectified
1.	Adding incompatible data types.	Rectified
2.	Entering a non-integer key.	Rectified.

Program: Hashing using linear probing.

using linear probing.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/week10/using line...

File Edit Format Run Options Window Help

```
size = 10
table = [-1] * size

def insert(key):
    index = key % size

    while table[index] != -1:
        index = (index + 1) % size

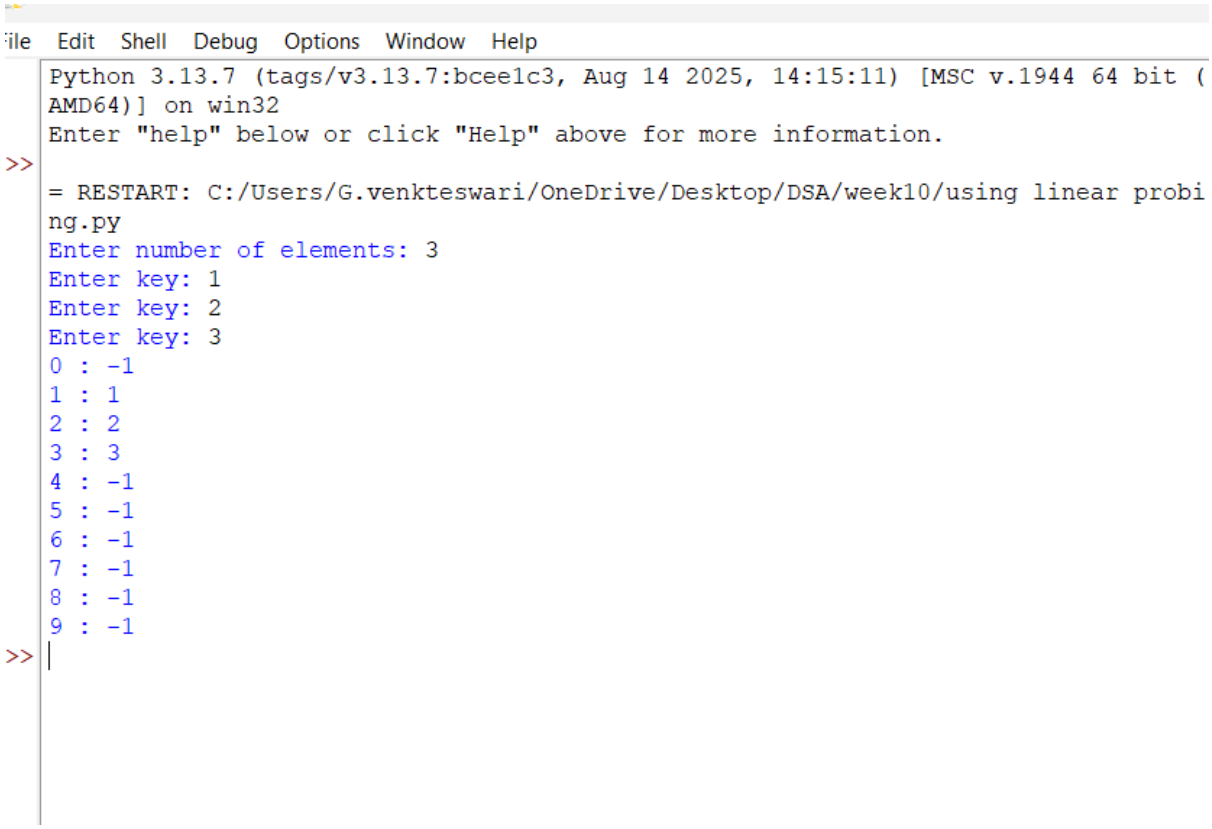
    table[index] = key

def display():
    for i in range(size):
        print(i, ":", table[i])

n = int(input("Enter number of elements: "))
for i in range(n):
    key = int(input("Enter key: "))
    insert(key)

display()
```

Output:



Explanation:

- The size = 10 statement defines the size of the hash table.
- The table = [-1] * size statement creates an empty table, where -1 represents an empty position.
- The insert() function calculates the index using key % size.
- If the position is occupied, the while loop checks the next position using (index + 1) % size.
- The display() function prints all indices and their values.
- The for loop accepts keys from the user and inserts them into the hash table using linear probing.

Error table:

S.no	Errors	Rectified
1.	Using an undefined variable or function.	Rectified
2.	Entering invalid input.	Rectified.

Program: Hashing using quadratic probing.

using quadratic probinh.py - C:/Users/G.venkteswari/OneDrive/Desktop/DSA/week10/using...

File Edit Format Run Options Window Help

```
size = 10
table = [-1] * size

def insert(key):
    index = key % size
    i = 0

    while i < size:
        new_index = (index + i * i) % size

        if table[new_index] == -1:
            table[new_index] = key
            return

        i += 1

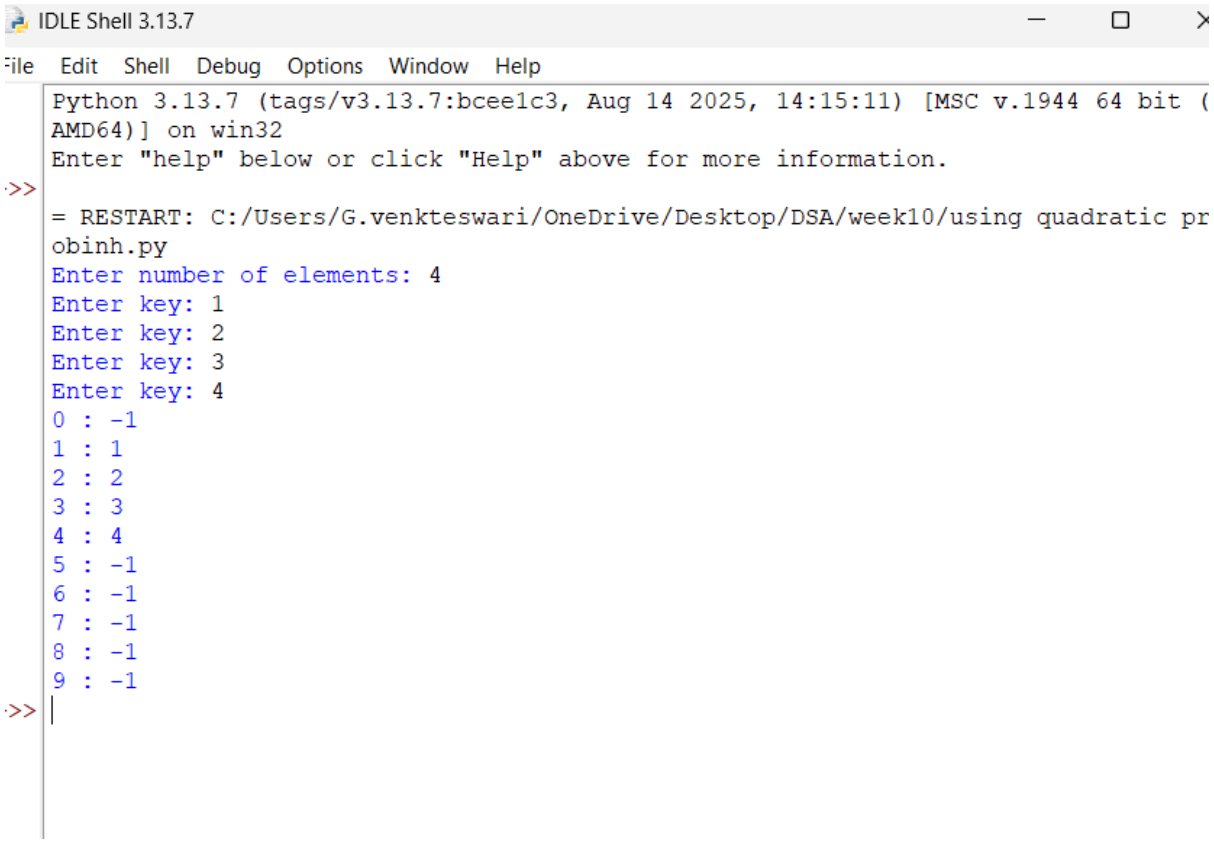
    print("Hash table is full or no slot found")

def display():
    for i in range(size):
        print(i, ":", table[i])

n = int(input("Enter number of elements: "))
for i in range(n):
    key = int(input("Enter key: "))
    insert(key)

display()
```

Output:



Explanation:

- The size = 10 statement defines the size of the hash table.
- The table = [-1] * size statement creates an empty table, where -1 indicates an empty position.
- The insert() function calculates the initial index using key % size.
- The while loop checks positions using (index + i*i) % size when a collision occurs.
- If an empty position is found, the key is inserted there; otherwise, the loop continues checking positions up to the table size.
- The display() function prints the hash table, while the for loop accepts keys from the user and inserts them.

Error table:

S.no	Errors	Rectified
1.	Using an undefined variable or function.	Rectified
2.	Keys are inserted successfully.	Rectified.